



第 9 章 控制单元的功能

系统结构研究所 · 计算机组成原理



第 9 章 控制单元的功能

9.1 操作命令的分析

9.2 控制单元的功能

9.1 操作命令的分析

完成一条指令分 4 个工作周期

取指周期

间址周期

执行周期

中断周期

一、取指周期

$PC \rightarrow MAR \rightarrow \text{地址线}$

$1 \rightarrow R$ (启动主存读操作)

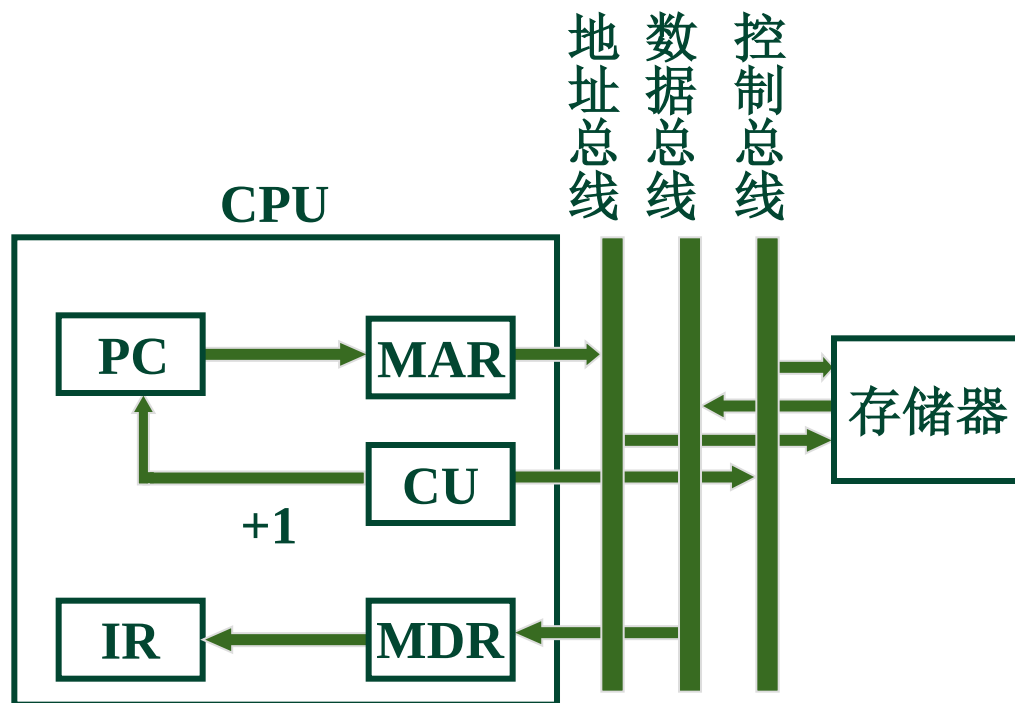
$M(MAR) \rightarrow MDR$

(将MAR所指的主存单元的内容读至MDR)

$MDR \rightarrow IR$

$OP(IR) \rightarrow CU$

$(PC) + 1 \rightarrow PC$



二、间址周期

9.1

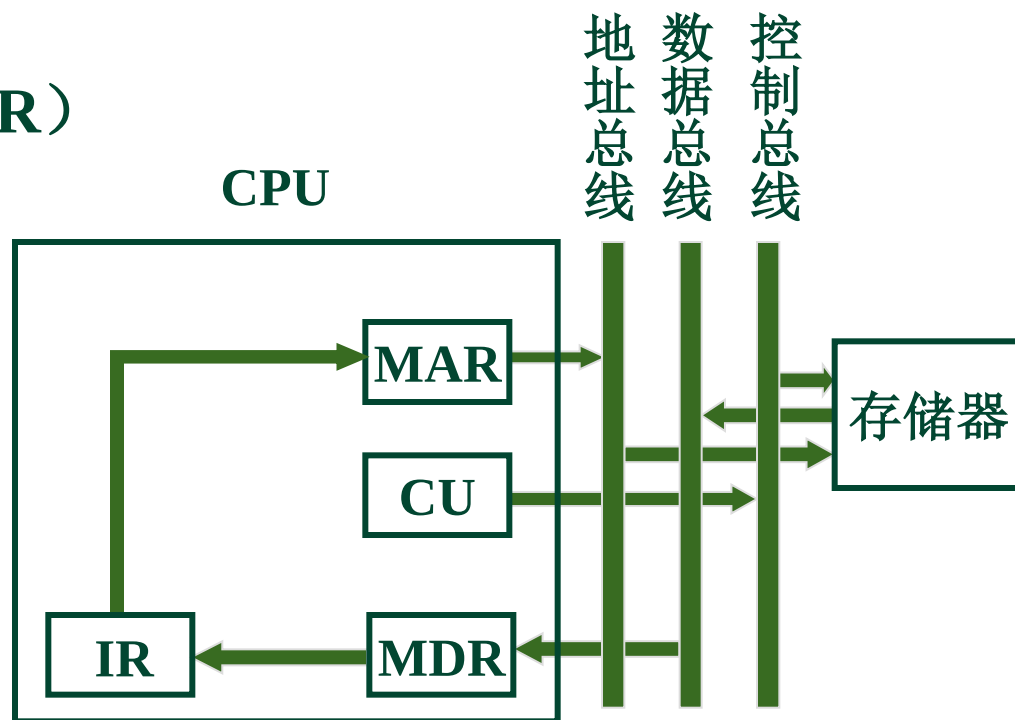
$Ad(IR) \rightarrow MAR$

(指令形式地址 $\rightarrow$ MAR)

$1 \rightarrow R$

$M(MAR) \rightarrow MDR$

$MDR \rightarrow Ad(IR)$



1. 非访存指令

(1) **CLA** 清A $0 \rightarrow \text{ACC}$

(2) **COM** 取反 $\overline{\text{ACC}} \rightarrow \text{ACC}$

(3) **SHR** 算术右移 $\text{L}(\text{ACC}) \rightarrow \text{R}(\text{ACC}), (\text{ACC}_0 \rightarrow \text{ACC}_0)$

(4) **CSL** 循环左移 $\text{R}(\text{ACC}) \rightarrow \text{L}(\text{ACC}), (\text{ACC}_0 \rightarrow \text{ACC}_n)$

(5) **STP** 停机指令 $0 \rightarrow \text{G}$ (运行标志触发器)

2. 访存指令

(1) 加法指令

ADD X

$\text{Ad(IR)} \rightarrow \text{MAR}$

$1 \rightarrow \text{R}$

$\text{M(MAR)} \rightarrow \text{MDR}$

$(\text{ACC}) + (\text{MDR}) \rightarrow \text{ACC}$

(2) 存数指令

STA X

$\text{Ad(IR)} \rightarrow \text{MAR}$

$1 \rightarrow \text{W}$

$\text{ACC} \rightarrow \text{MDR}$

$\text{MDR} \rightarrow \text{M(MAR)}$

(3) 取数指令

LDA X $\text{Ad}(\text{IR}) \rightarrow \text{MAR}$ $1 \rightarrow \text{R}$ $\text{M}(\text{MAR}) \rightarrow \text{MDR}$ $\text{MDR} \rightarrow \text{ACC}$

3. 转移指令

(1) 无条件转

JMP X $\text{Ad}(\text{IR}) \rightarrow \text{PC}$

(2) 条件转移

BAN X (负则转) $A_0 \cdot \text{Ad}(\text{IR}) + \bar{A}_0(\text{PC}) \rightarrow \text{PC}$ (结果为负即 $A_0=1$)

4. 三类指令的指令周期



四、中断周期

9.1

程序断点存入 “0” 地址 程序断点 进栈

$0 \rightarrow \text{MAR}$

$(\text{SP}) - 1 \rightarrow \text{MAR}$

$1 \rightarrow \text{W}$

$1 \rightarrow \text{W}$

$\text{PC} \rightarrow \text{MDR}$

$\text{PC} \rightarrow \text{MDR}$

$\text{MDR} \rightarrow \text{M}(\text{MAR})$

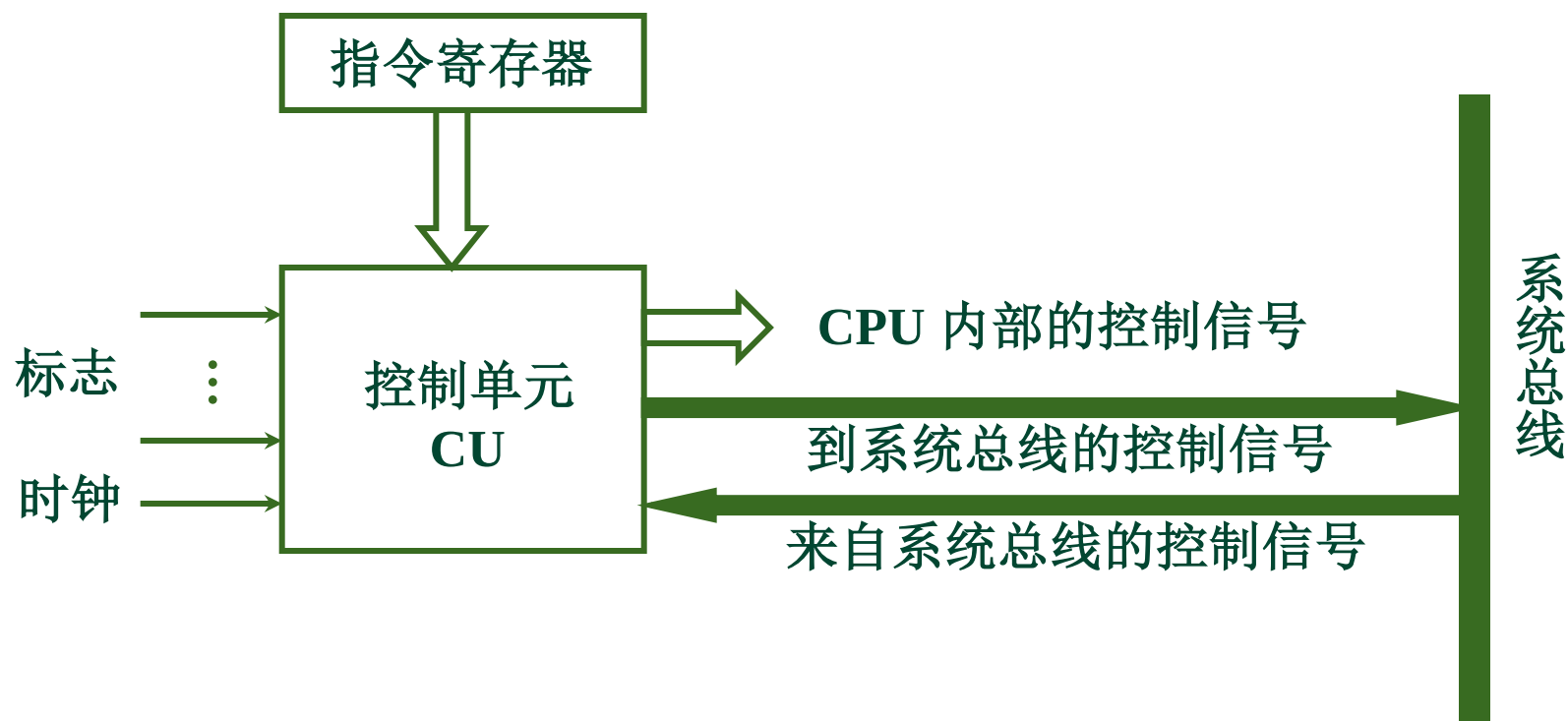
$\text{MDR} \rightarrow \text{M}(\text{MAR})$

中断识别程序入口地址 $\text{M} \rightarrow \text{PC}$

$0 \rightarrow \text{EINT} \text{ (置 “0”)}$

$0 \rightarrow \text{EINT} \text{ (置 “0”)}$

一、控制单元的外特性



1. 输入信号

9.2

(1) 时钟

CU 受时钟控制

一个时钟脉冲

发一个操作命令或一组需同时执行的操作命令

(2) 指令寄存器 $OP(IR) \rightarrow CU$

控制信号 与操作码有关

(3) 标志

CU 受标志控制

(4) 外来信号

如 **INTR** 中断请求

HRQ 总线请求

2. 输出信号

(1) CPU 内的各种控制信号

$R_i \rightarrow R_j$

$(PC) + 1 \rightarrow PC$

ALU +、-、与、或

(2) 送至控制总线的信号

$\overline{\text{MREQ}}$

访存控制信号

$\overline{\text{IO/M}}$

访 IO/ 存储器的控制信号

$\overline{\text{RD}}$

读命令

$\overline{\text{WR}}$

写命令

INTA

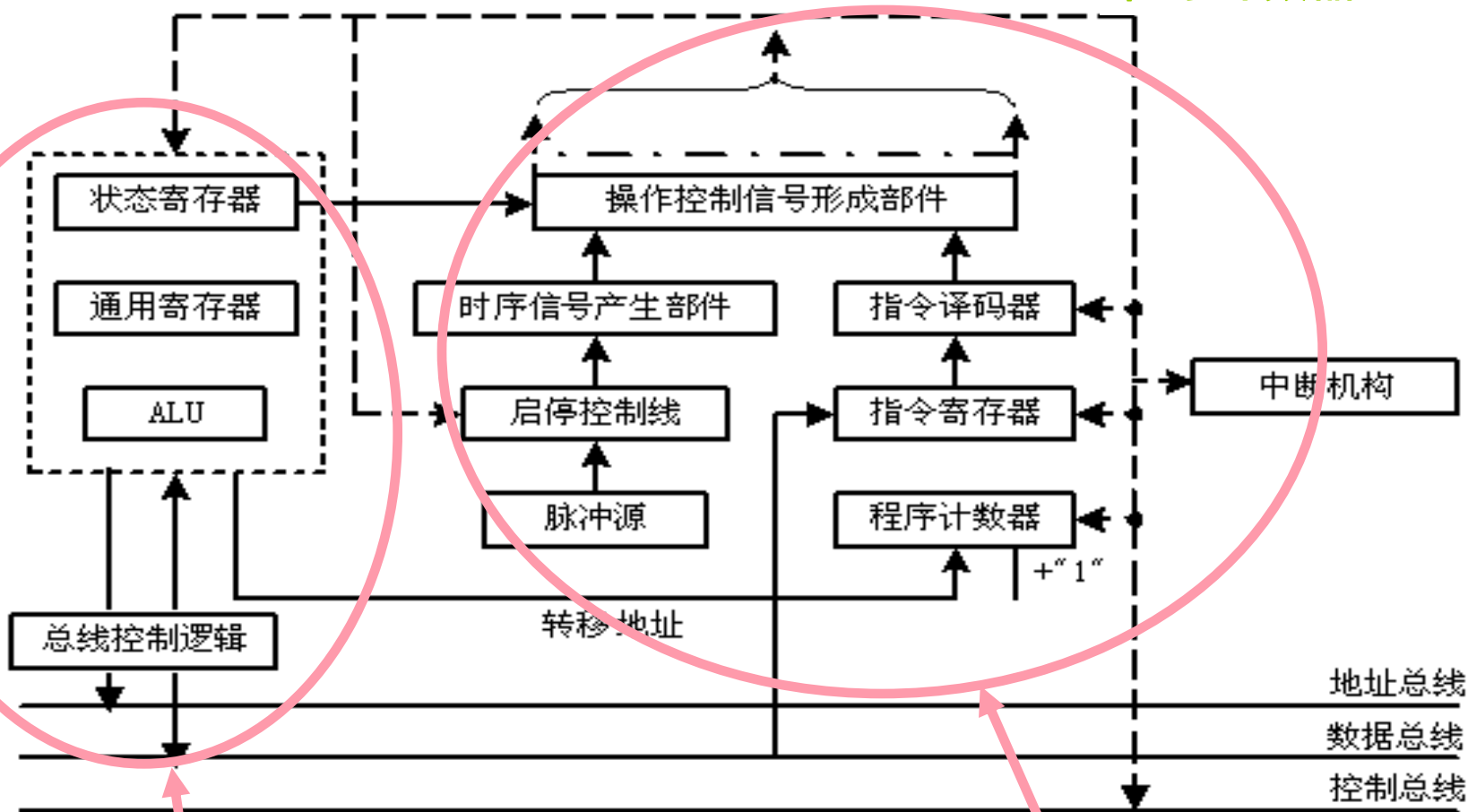
中断响应信号

HLDA

总线响应信号

CPU基本组成原理图

指令寄存器---IR
程序计数器---PC



执行部件

控制部件

CPU 由 执行部件 和 控制部件 组成
CPU 包含 数据通路 和 控制器

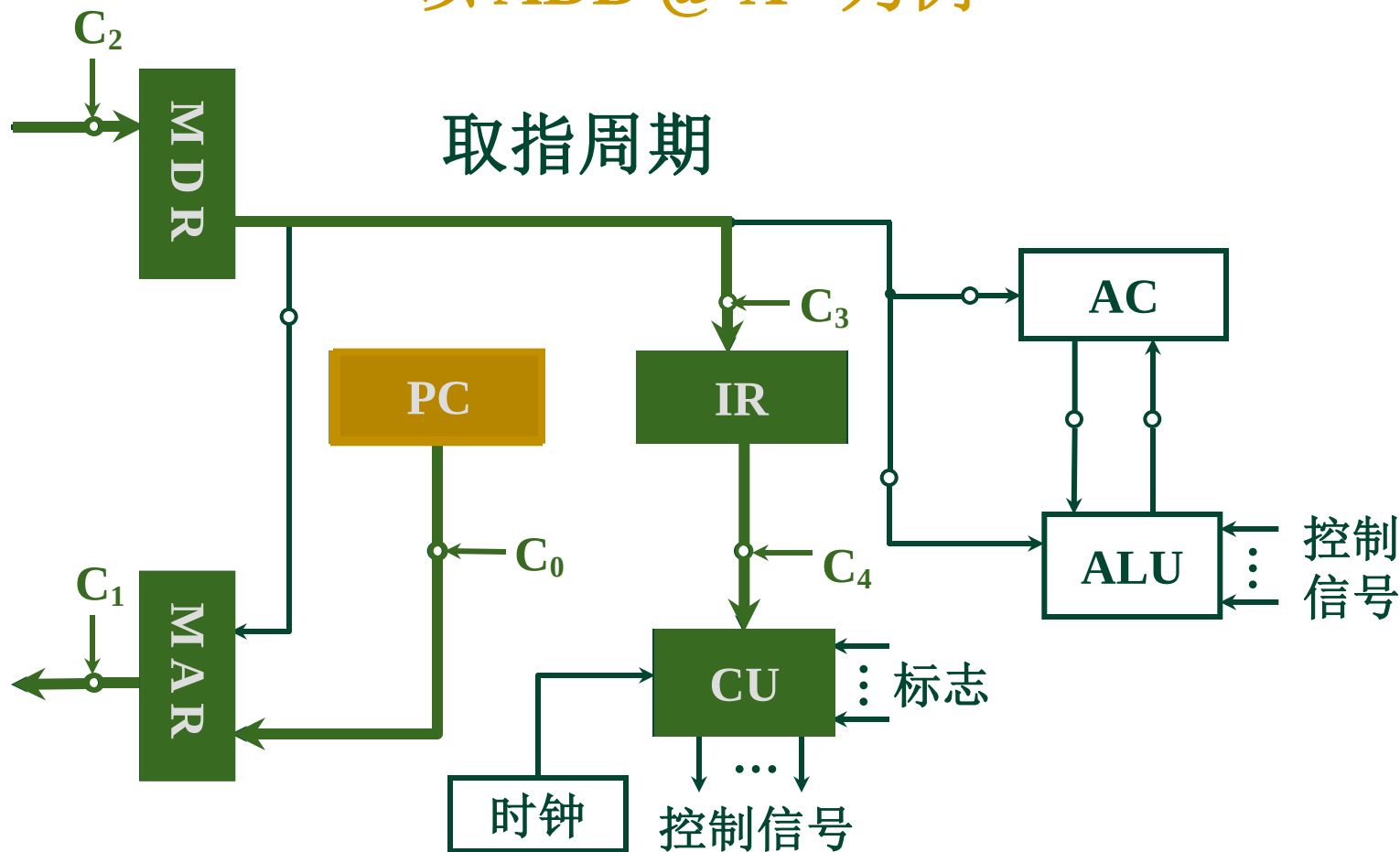
控制器 由 指令译码器 和 控制信号形成部件 组成

二、控制信号举例

9.2

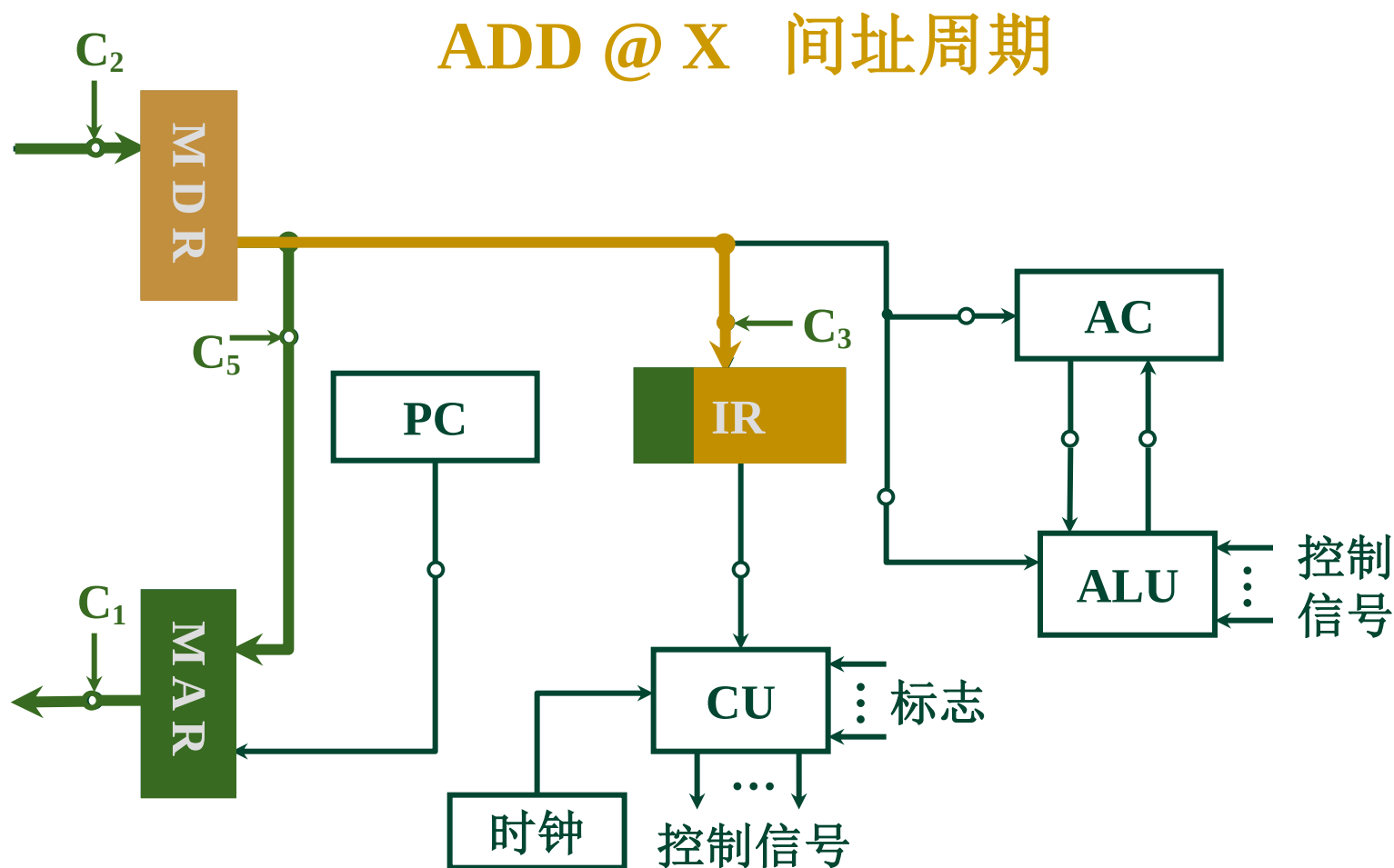
1. 不采用 CPU 内部总线的方式

以 **ADD @ X** 为例



二、控制信号举例

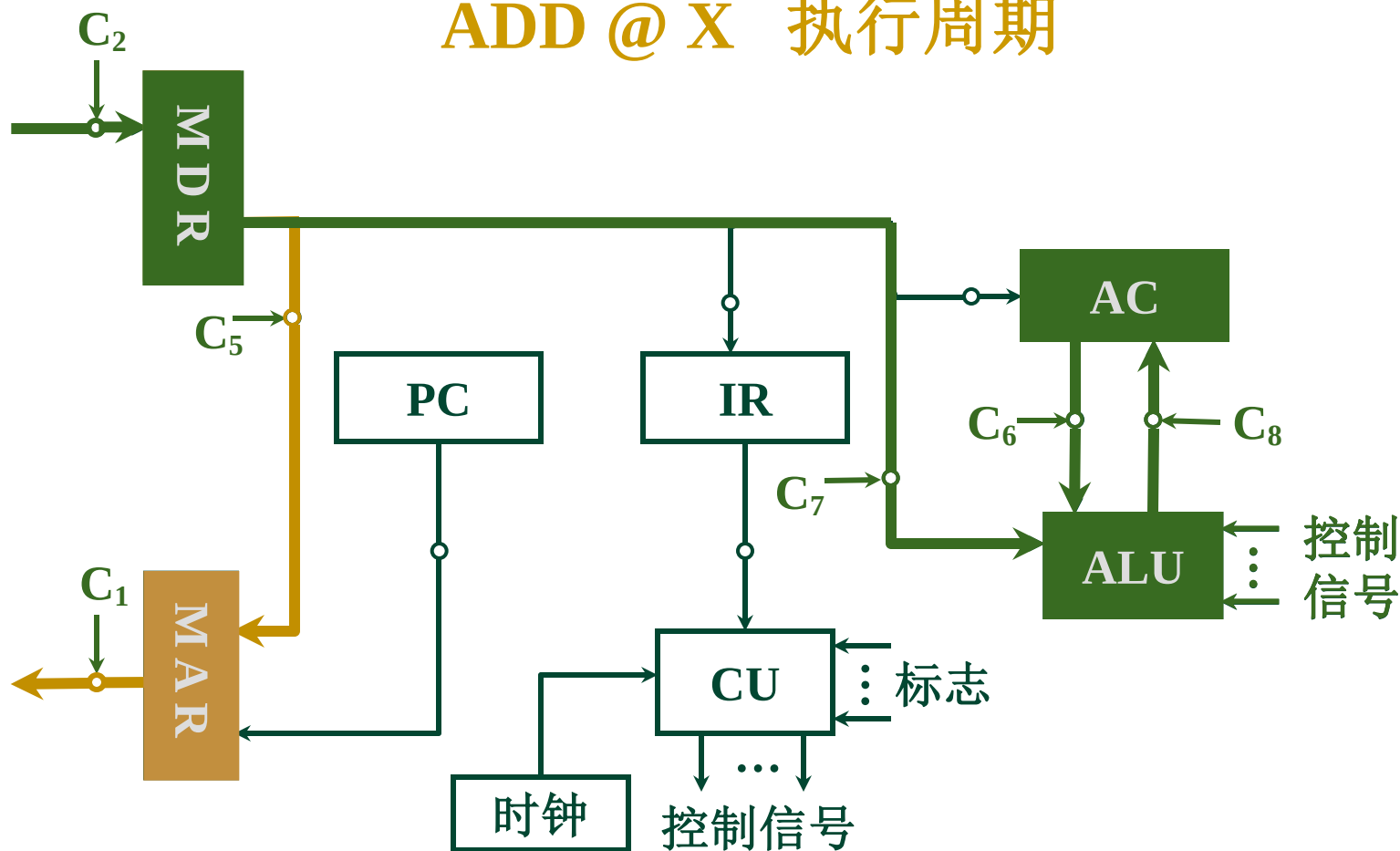
1. 不采用 CPU 内部总线的方式



二、控制信号举例

1. 不采用 CPU 内部总线的方式

ADD @ X 执行周期



2. 采用 CPU 内部总线方式

(1) ADD @ X 取指周期

• $PC \longrightarrow MAR \longrightarrow \text{地址线}$

$PC_0 \quad MAR_i$

• CU 发读命令 $1 \longrightarrow R$

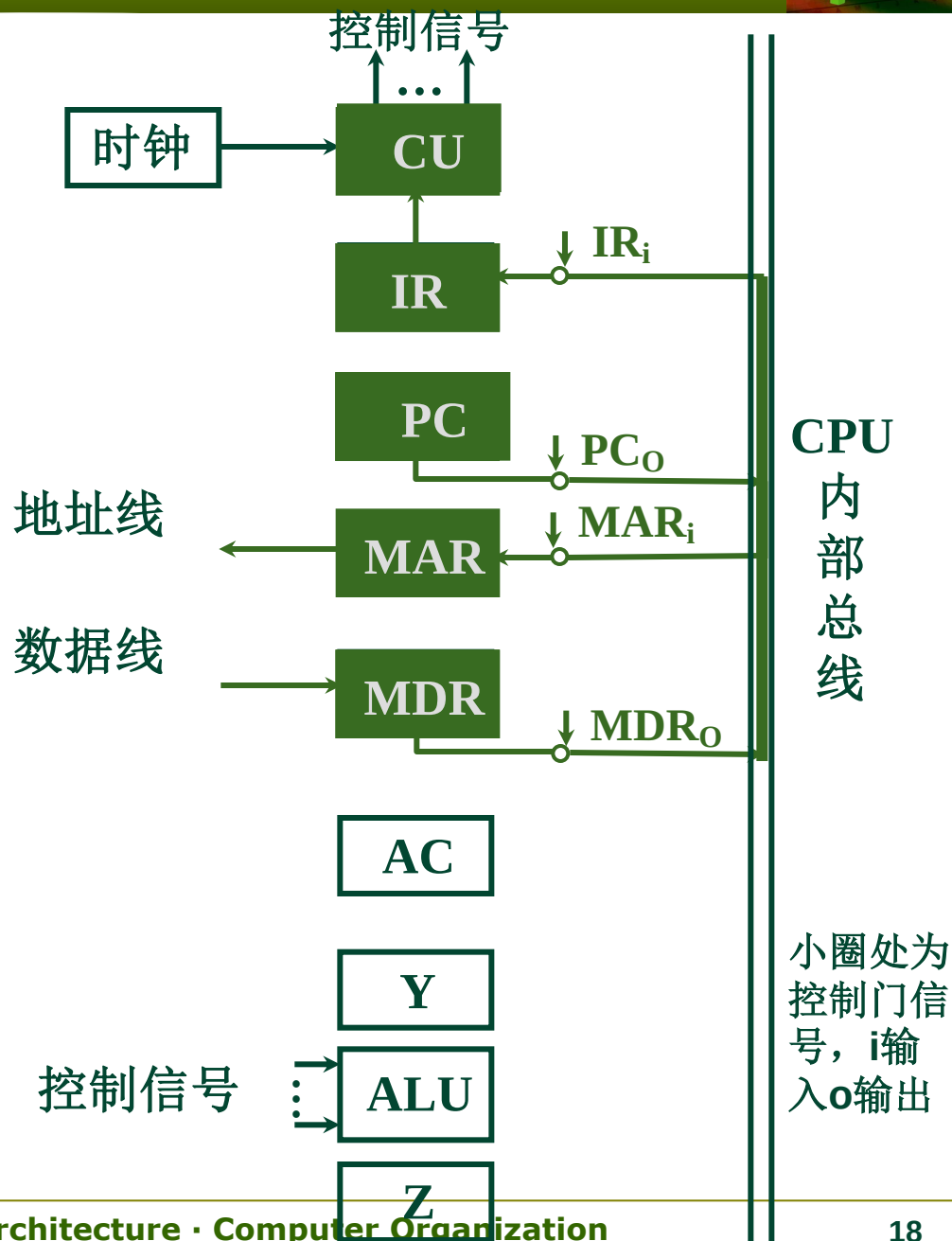
• 数据线 $\longrightarrow MDR$

• $MDR \longrightarrow IR$

$MDR_0 \quad IR_i$

• $OP(IR) \longrightarrow CU$

• $(PC) + 1 \longrightarrow PC$



(2) ADD @ X 间址周期

形式地址 $\rightarrow$ MAR

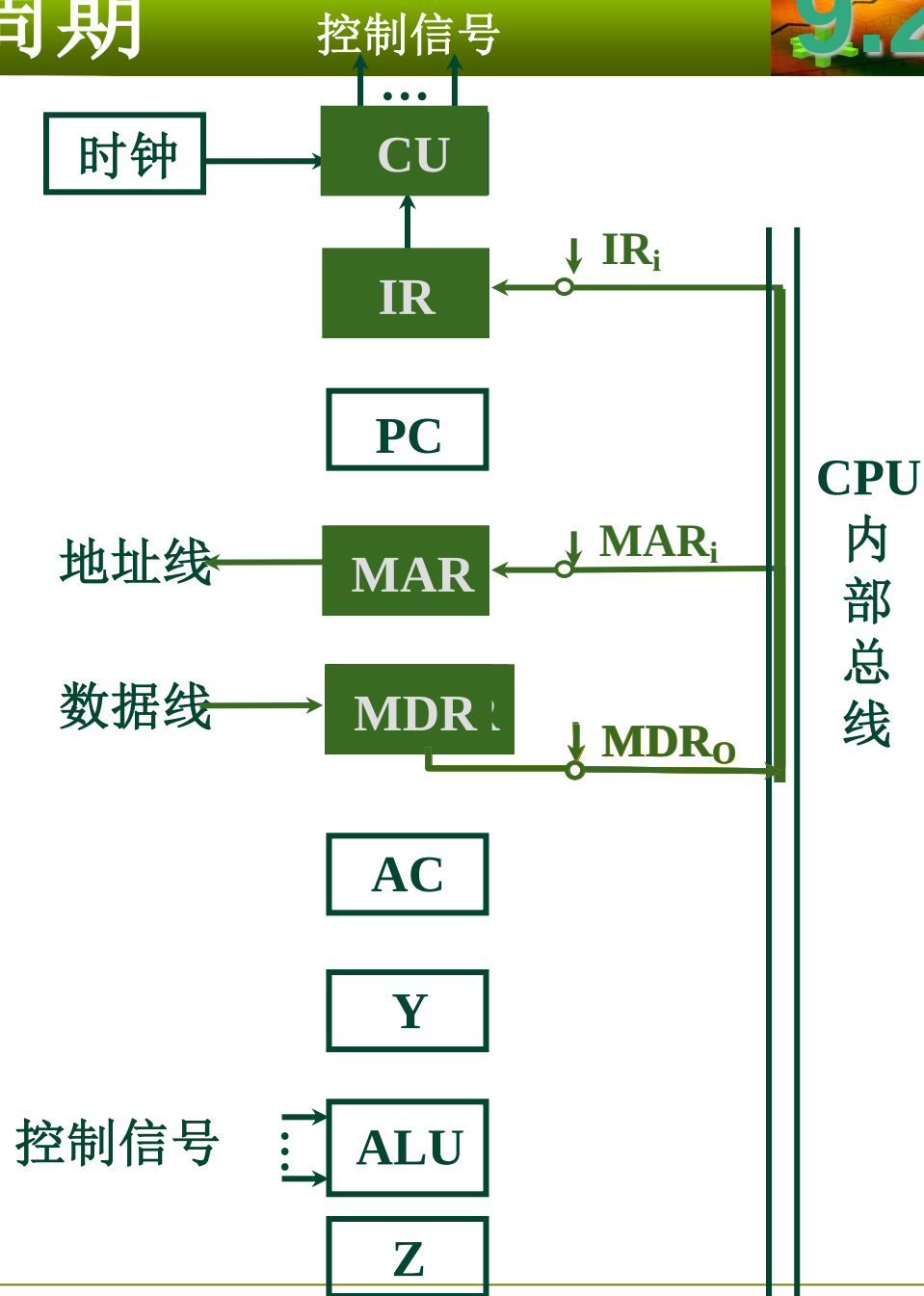
• $\text{MDR} \rightarrow \text{MAR} \rightarrow \text{地址线}$
 $\text{MDR}_0 \quad \text{MAR}_i$

• $1 \rightarrow R$

• 数据线 $\rightarrow$ MDR

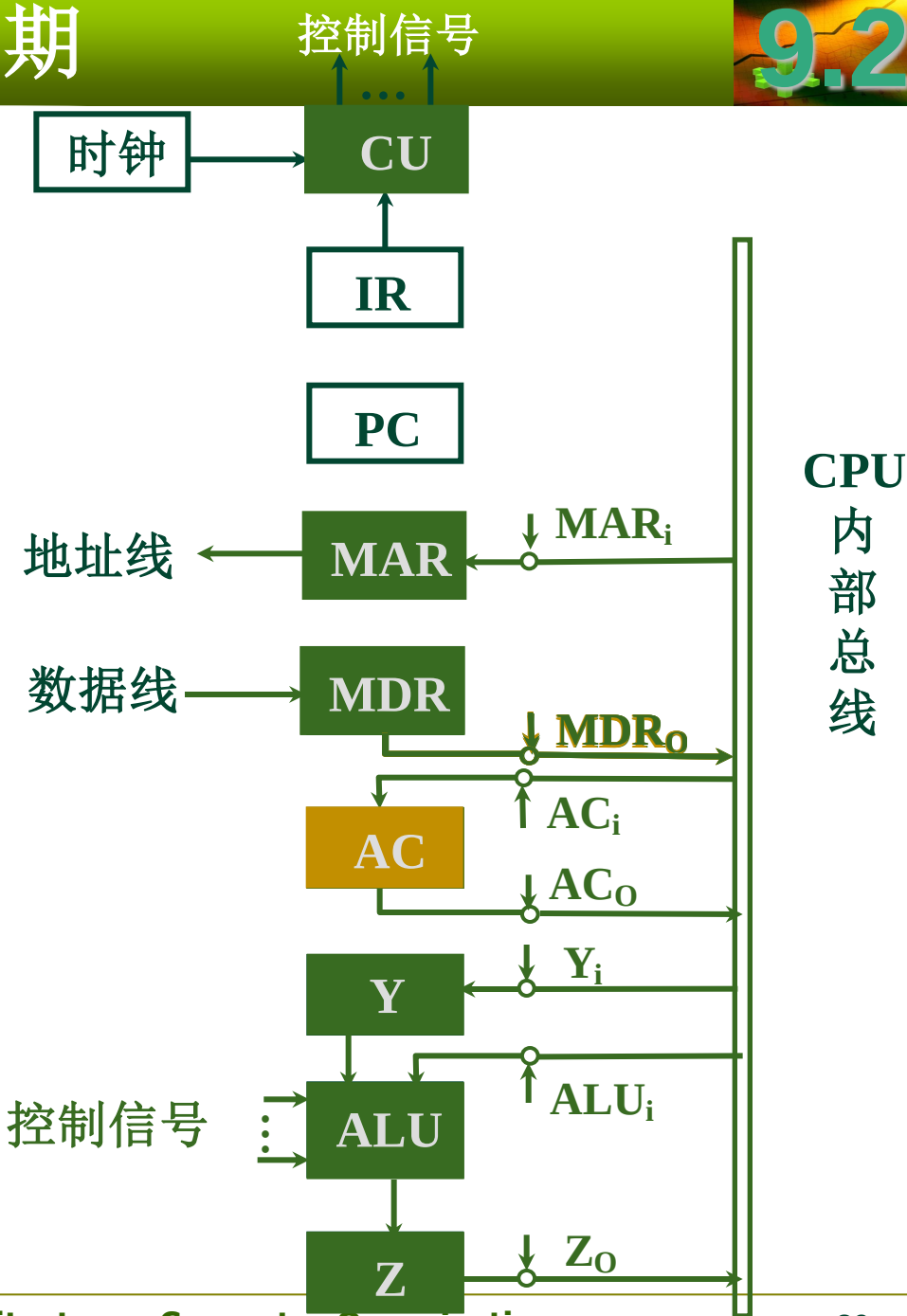
• $\text{MDR} \rightarrow \text{IR}$
 $\text{MDR}_0 \quad \text{IR}_i$

有效地址 $\rightarrow \text{Ad}(\text{IR})$



(3) ADD @ X 执行周期

- $\text{MDR} \rightarrow \text{MAR} \rightarrow \text{地址线}$
 $\text{MDR}_0 \quad \text{MAR}_i$
- $1 \rightarrow R$
- 数据线 $\rightarrow \text{MDR}$
- $\text{MDR} \rightarrow Y \rightarrow \text{ALU}$
 $\text{MDR}_0 \quad Y_i$
- $\text{AC} \rightarrow \text{ALU}$
 $\text{AC}_0 \quad \text{ALU}_i$
- $(\text{AC}) + (\text{Y}) \rightarrow Z$
- $Z \rightarrow \text{AC}$
 $Z_0 \quad \text{AC}_i$

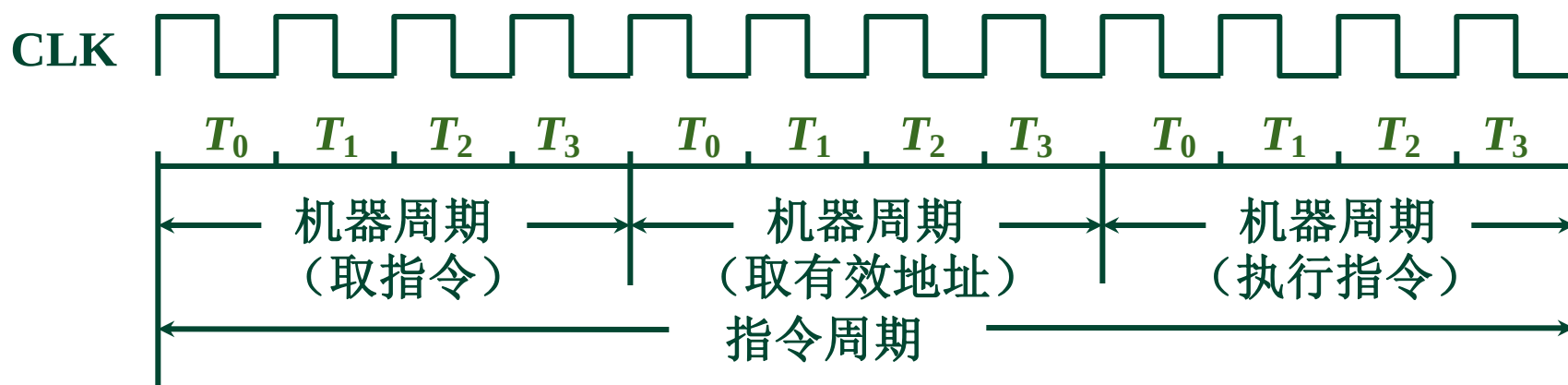


四、CU的控制方式

产生不同微操作命令序列所用的时序控制方式

1. 同步控制方式

任一微操作均由 **统一基准时标** 的时序信号控制



(1) 采用 **定长** 的机器周期

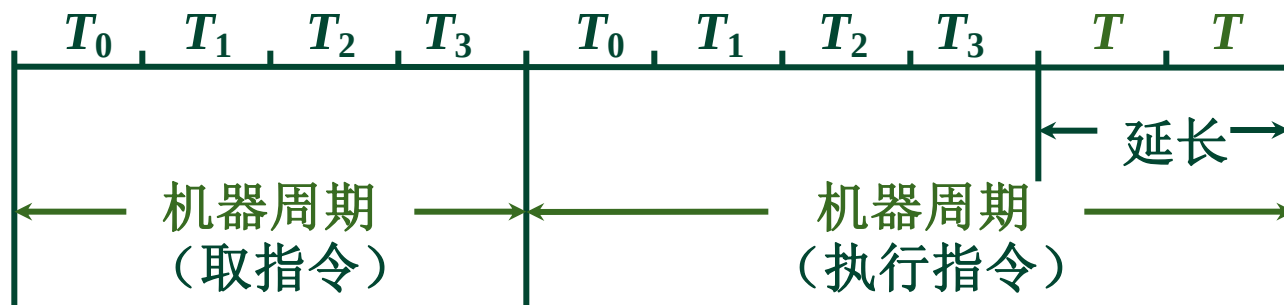
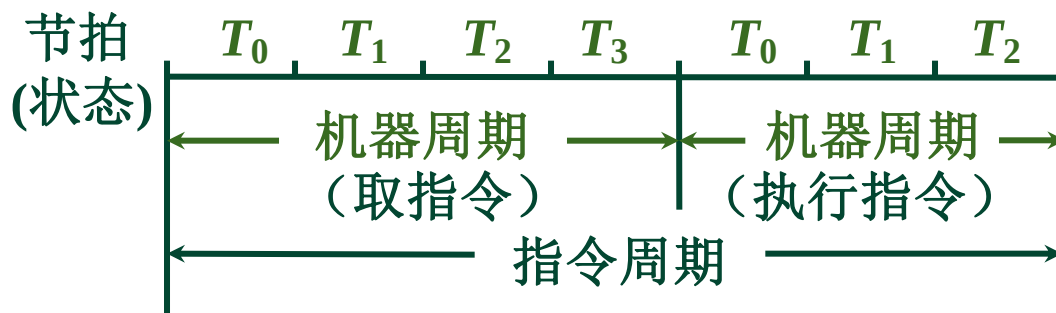
以 **最长** 的 **微操作序列** 和 **最繁** 的微操作作为 **标准**

机器周期内 **节拍数相同**

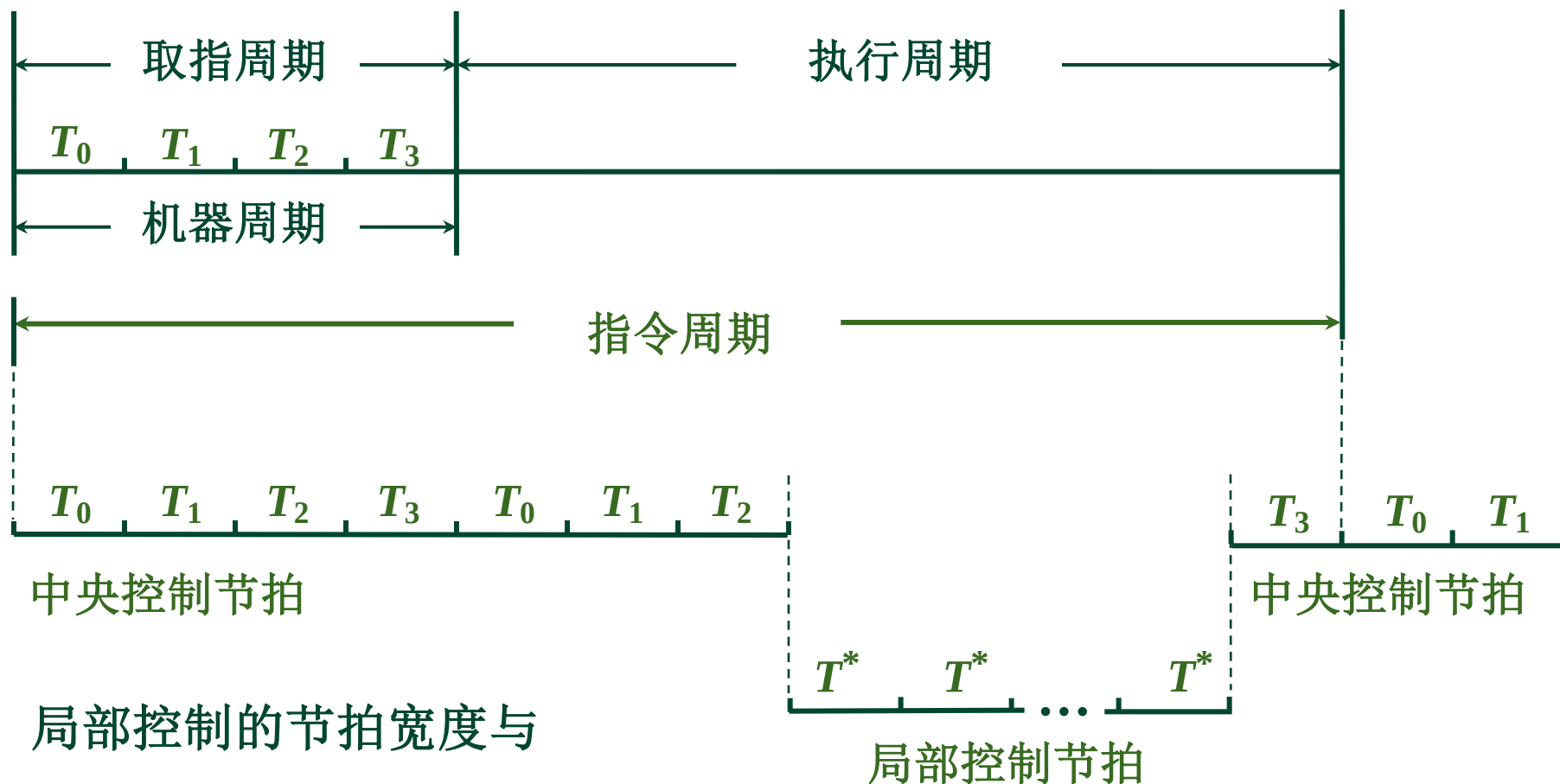
(2) 采用不定长的机器周期

9.2

机器周期内 节拍数不等



(3) 采用中央控制和局部控制相结合的方法



局部控制的节拍宽度与
中央控制的节拍宽度一致

例.设某机平均执行一条指令需要两次访问内存，平均需要3个CPU周期，每个CPU周期平均包含4个节拍周期。若机器主频为240MHz，问：

(1) 若主存为“0等待”（即不需要插入等待周期），问执行一条指令的平均时间为多少？

(2) 若每次访问内存需要插入2个等待周期，问执行一条指令的平均时间又是多少？

解：因为主频为240MHz，所以节拍周期= $(1/240) \mu s$ 每个
因为每个CPU周期平均包含4个节拍周期，所以：

$$\text{CPU周期} = \text{节拍周期} \times 4 = 4/240\text{MHz} = (1/60)\mu s$$

若访存不需要插入等待周期，则执行一条指令平均需要3个CPU周期，所以：

$$\text{指令周期} = 3 \times \text{CPU周期} = 3 \times (1/60) \mu s = (1/20)\mu s = 0.05\mu s$$

$$\text{机器平均速度} = 1/0.05\mu s = 20 \text{ MIPS}$$

(2) 平均执行一条指令需要两次访问内存，每次访问内存需要插入2个等待周期，所以：

$$\begin{aligned} \text{指令周期} &= 0.05\mu s + 2 \times (1/240)\mu s = (1/20)\mu s + (1/120)\mu s \\ &= (7/120)\mu s \end{aligned}$$

$$\text{机器平均速度} = 120/7 \approx 17 \text{ MIPS}$$

$$\frac{\text{MIPS}_1}{\text{MIPS}_2} = \frac{f_1}{f_2}$$

若某机主频为200MHz，每个指令周期平均为2.5CPU周期，每个CPU周期平均包括2个主频周期，问：

(1)该机平均指令执行速度为多少MIPS？

(2)若主频不变，但每条指令平均包括5个CPU周期，每个CPU周期又包含4个主频周期，平均指令执行速度又为多少MIPS？
由此可得出什么结论？

解：（1）主频为200MHz，所以主频周期 =
 $1/200\text{MHz} = 0.005\mu\text{s}$

每个指令周期平均为2.5CPU周期，每个CPU周期平均包括2个主频周期，所以一条指令的执行时间 = $2.5 \times 2 \times 0.005\mu\text{s} = 0.025\mu\text{s}$

该机平均指令执行速度 = $1/0.025 = 40\text{MIPS}$ 。

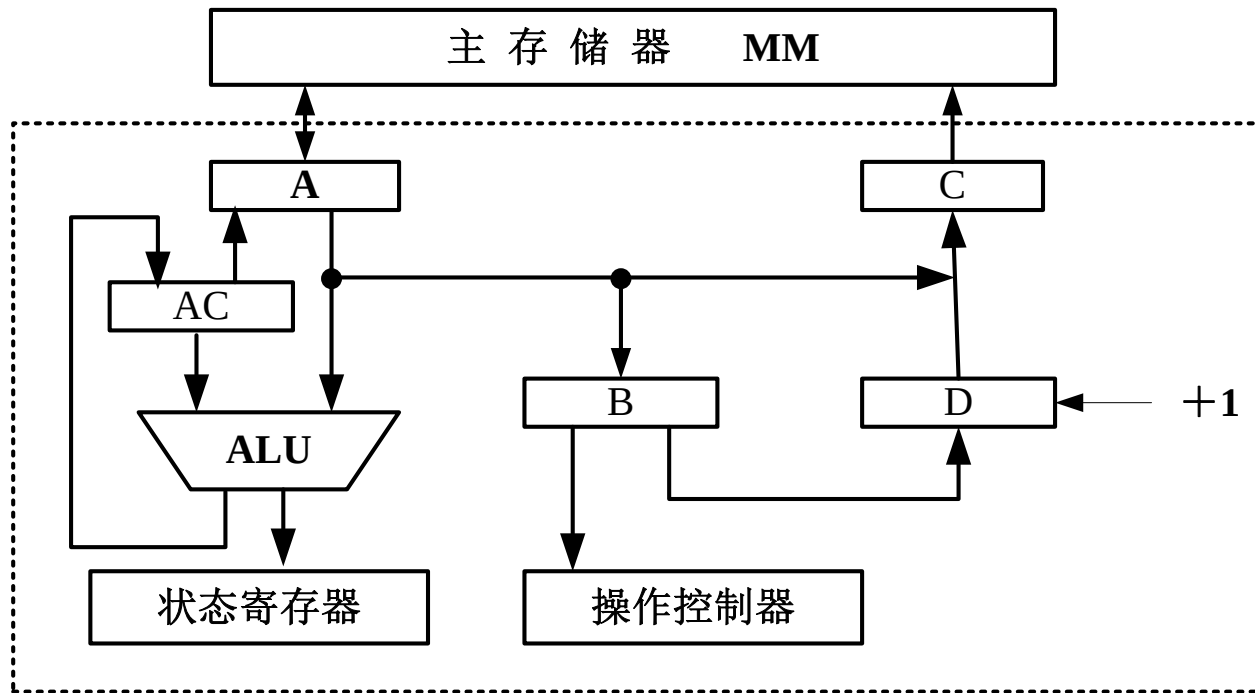
(2) 每条指令平均包括5个CPU周期，每个CPU周期又包含4个主频周期，所以一条指令的执行时间 = $4 \times 5 \times 0.005\mu\text{s} = 0.1\mu\text{s}$

该机平均指令执行速度 = $1/0.1 = 10\text{MIPS}$

(3)说明指令的复杂程度会影响指令的平均执行速度。

例. CPU结构如图所示，其中包括一个累加寄存器AC、一个状态寄存器和其他四个寄存器，各部分之间的连线表示数据通路，箭头表示信息传送方向

(1) 标明图中四个寄存器的名称



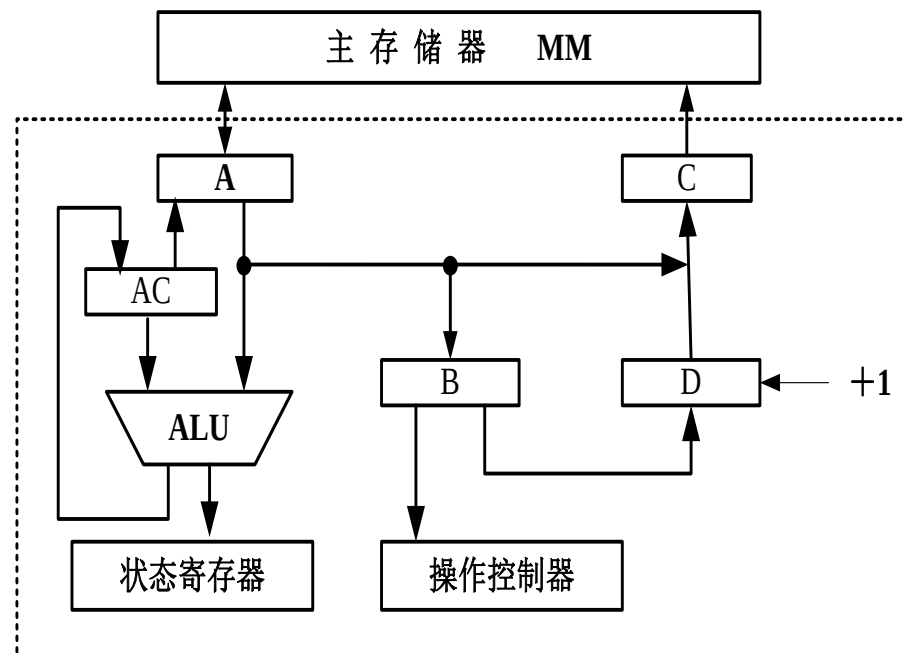
解：A为MDR，B为IR，C为MAR，D为PC

(2) 简述取指令的数据通路

(3) 简述完成指令LDA X的数据通路 (X为内存地址, LDA的功能为 $(X) \rightarrow (AC)$)

(4) 简述完成指令ADDY的数据通路 (Y为内存地址, ADD功能为 $(AC) + (Y) \rightarrow (AC)$)

(5) 简述完成指令STA Z的数据通路 (Z为内存地址, STA功能为 $(AC) \rightarrow (Z)$)



解: (2)取指: $PC \rightarrow MAR \rightarrow MM \rightarrow MDR \rightarrow IR$

(3) LDA X: $X \rightarrow MAR \rightarrow MM \rightarrow MDR \rightarrow ALU \rightarrow AC$

(4) ADD Y: $Y \rightarrow MAR \rightarrow MM \rightarrow MDR \rightarrow ALU \rightarrow ADD \rightarrow AC$

(5) STA Z: $Z \rightarrow MAR, AC \rightarrow MDR \rightarrow MM$

2. 异步控制方式

无基准时标信号

无固定的周期节拍和严格的时钟同步

采用 应答方式

3. 联合控制方式

同步与异步相结合

大部分统一、小部分区别对待

如：取指同步、I/O异步

4. 人工控制方式

(1) Reset

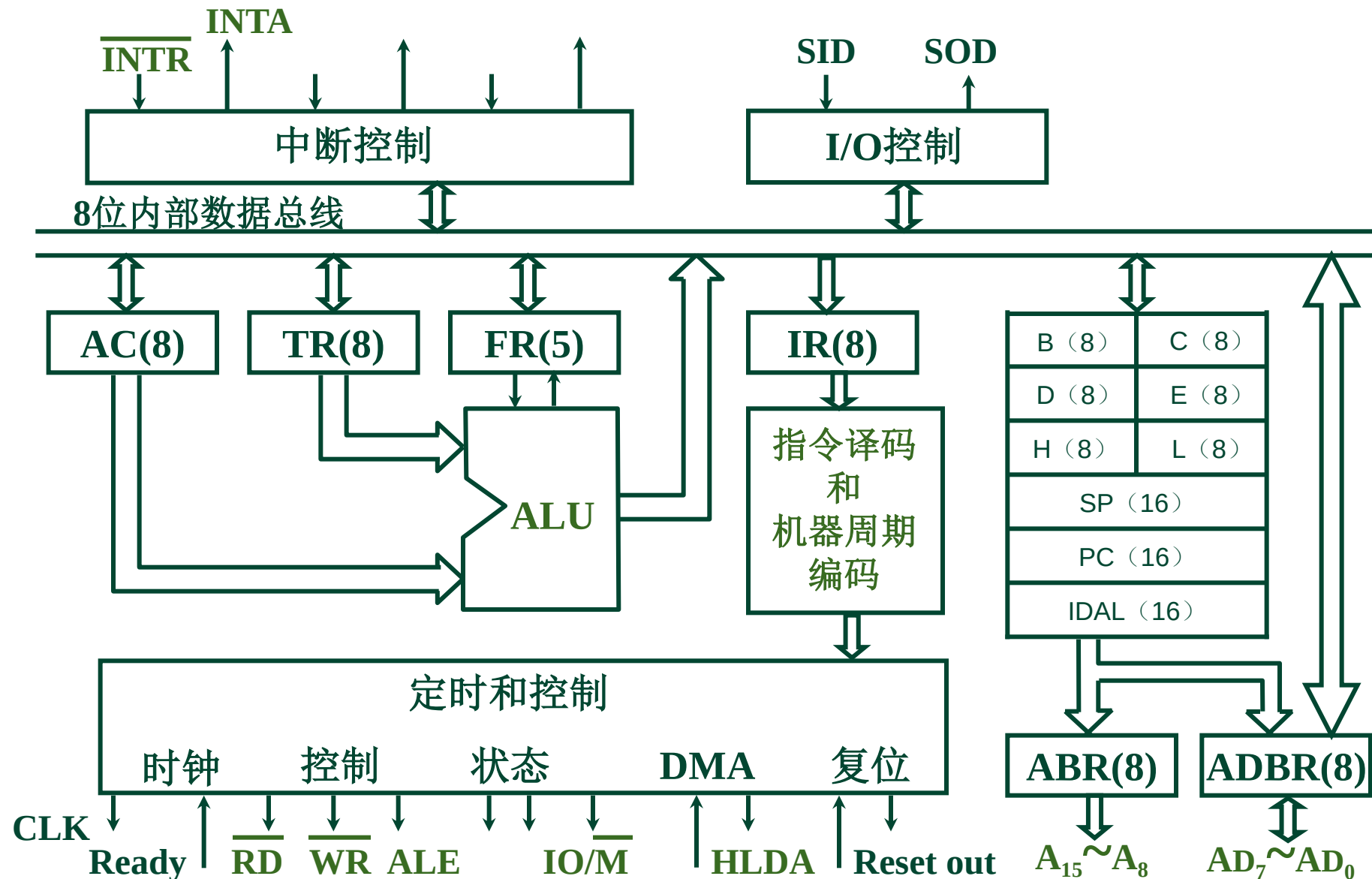
(2) 连续 和 单条 指令执行转换开关

(3) 符合停机开关

五、多级时序系统实例分析

9.2

1. 8085 的组成



2. 8085 的外部引脚

9.2

(1) 地址和数据信号

$A_{15} \sim A_8$ $AD_7 \sim AD_0$

SID SOD

(2) 定时和控制信号

入 X_1 X_2

出 CLK ALE S_0 S_1
 $IO/\overline{M}$ $\overline{RD}$ $\overline{WR}$

(3) 存储器和 I/O 初始化

入 HOLD Ready

出 HLDA

X_1	1	40	V_{CC}
X_2	2	39	HOLD
Reset out	3	38	HLDA
SOD	4	37	CLK(out)
SID	5	36	Rstest in
Trap	6	35	Ready
RST7.5	7	34	$IO/\overline{M}$
RST6.5	8	33	S_1
RST5.5	9	32	$\overline{RD}$
$\overline{INTR}$	10	31	$\overline{WR}$
INTA	11	30	ALE
AD_0	12	29	S_0
AD_1	13	28	A_{15}
AD_2	14	27	A_{14}
AD_3	15	26	A_{13}
AD_4	16	25	A_{12}
AD_5	17	24	A_{11}
AD_6	18	23	A_{10}
AD_7	19	22	A_9
V_{SS}	20	21	A_8

(4) 与中断有关的信号

9.2

入 $\overline{\text{INTR}}$

出 INTA

Trap 重新启动中断

(5) CPU 初始化

入 $\overline{\text{Reset in}}$

出 Reset out

(6) 电源和地

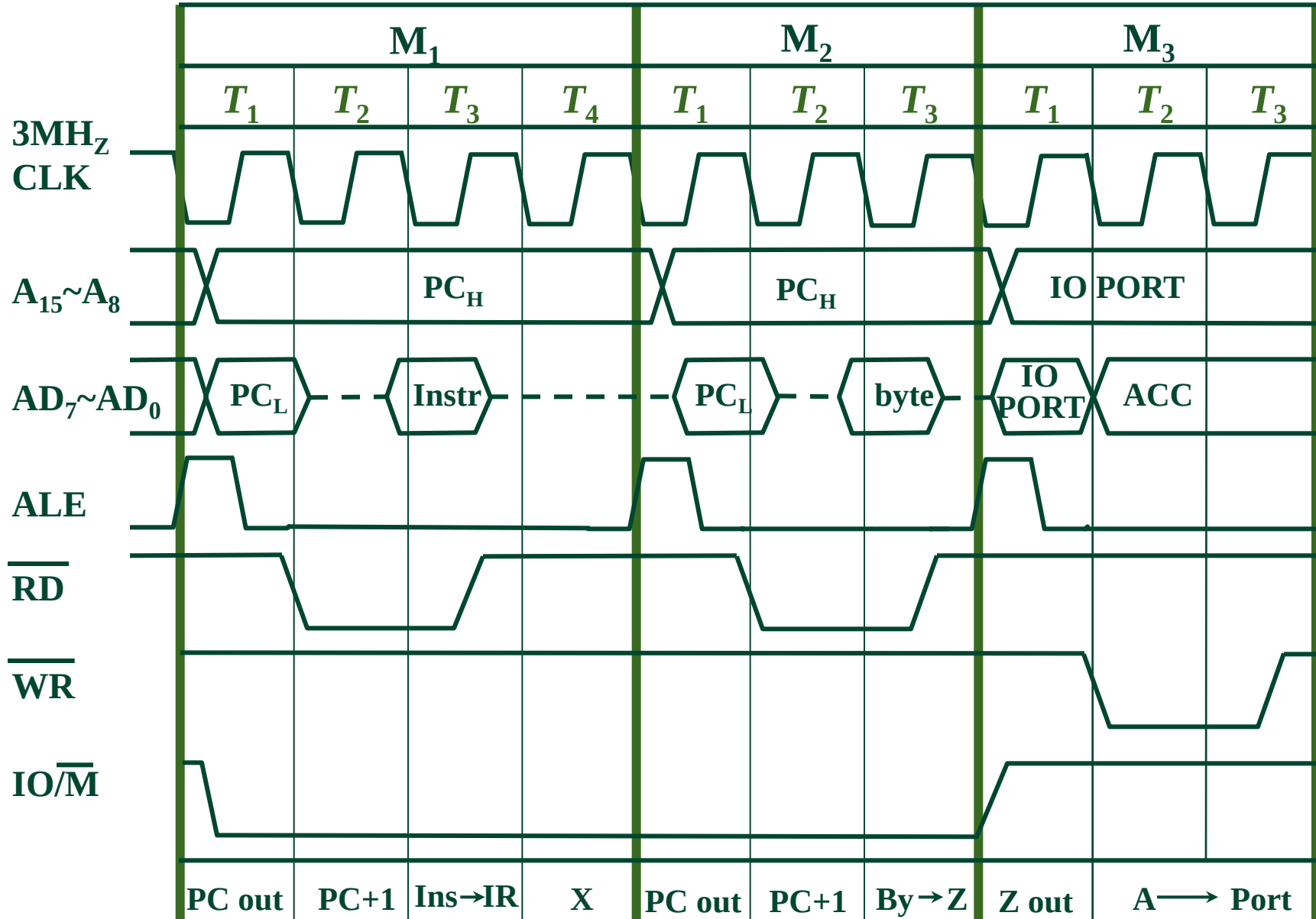
V_{CC} +5 V

V_{SS} 地

X_1	1	40	V_{CC}
X_2	2	39	HOLD
Reset out	3	38	HLDA
SOD	4	37	$\overline{\text{CLK(out)}}$
SID	5	36	$\overline{\text{Rreset in}}$
Trap	6	35	Ready
RST7.5	7	34	IO/M
RST6.5	8	33	$\overline{S_1}$
RST5.5	9	32	$\overline{\text{RD}}$
$\overline{\text{INTR}}$	10	31	$\overline{\text{WR}}$
INTA	11	30	ALE
AD_0	12	29	S_0
AD_1	13	28	A_{15}
AD_2	14	27	A_{14}
AD_3	15	26	A_{13}
AD_4	16	25	A_{12}
AD_5	17	24	A_{11}
AD_6	18	23	A_{10}
AD_7	19	22	A_9
V_{SS}	20	21	A_8



3. 机器周期和节拍（状态）与控制信号的关系



以一条输出指令（I/O 写）为例

机器周期 M_1 取指令操作码

机器周期 M_2 取设备地址

机器周期 M_3 执行 ACC 的内容写入设备

每个 控制 信号在 指定机器周期 的
指定节拍 T 时刻 发出



第10章 控制单元的设计

系统结构研究所·计算机组成原理



10.1 组合逻辑设计

10.2 微程序设计

- 组合逻辑型：核心是微操作产生部件，用组合逻辑设计思想，以布尔代数作为工具设计。输入信号来自指令译码器的输出、时序发生器的时序信号和程序运行结果特征及状态，输出为带有时间标志的微操作控制信号。
- 微程序控制型：将机器指令分解为基本微命令序列，用二进制码表示微命令，并编成微指令，多条微指令形成微程序。每种机器指令对应一段微程序，在制造**CPU**时固化在**CPU**中的一个控制存储器中（**CS**或**CM**）中。执行一条机器指令时，**CPU**依次从**CS**中取微指令，从而产生微命令。

基本概念

- 微命令：微程序控制计算机中的微操作控制信号。
- 微操作：控制器中执行部件接受微指令后所进行的操作，是指令序列中最基本、不可分割的动作。
 - 所有的微操作要在一个机器周期完成
 - 例如，一条加法指令要分成四步完成：取指令，计算地址，取数，加法运算，每一步要实现若干个微操作
- 微指令：在微程序控制的计算机中，同时发出的控制信号所执行的一组微操作称为微指令。
 - 是在机器的一个节拍中，一组实现一定操作功能的微命令
 - 将一条指令分解成若干条微指令，按次序执行微指令，即可实现指令的功能
- 微程序：由微指令组成的序列称为微程序。一个微程序的功能对应一条机器指令的功能。

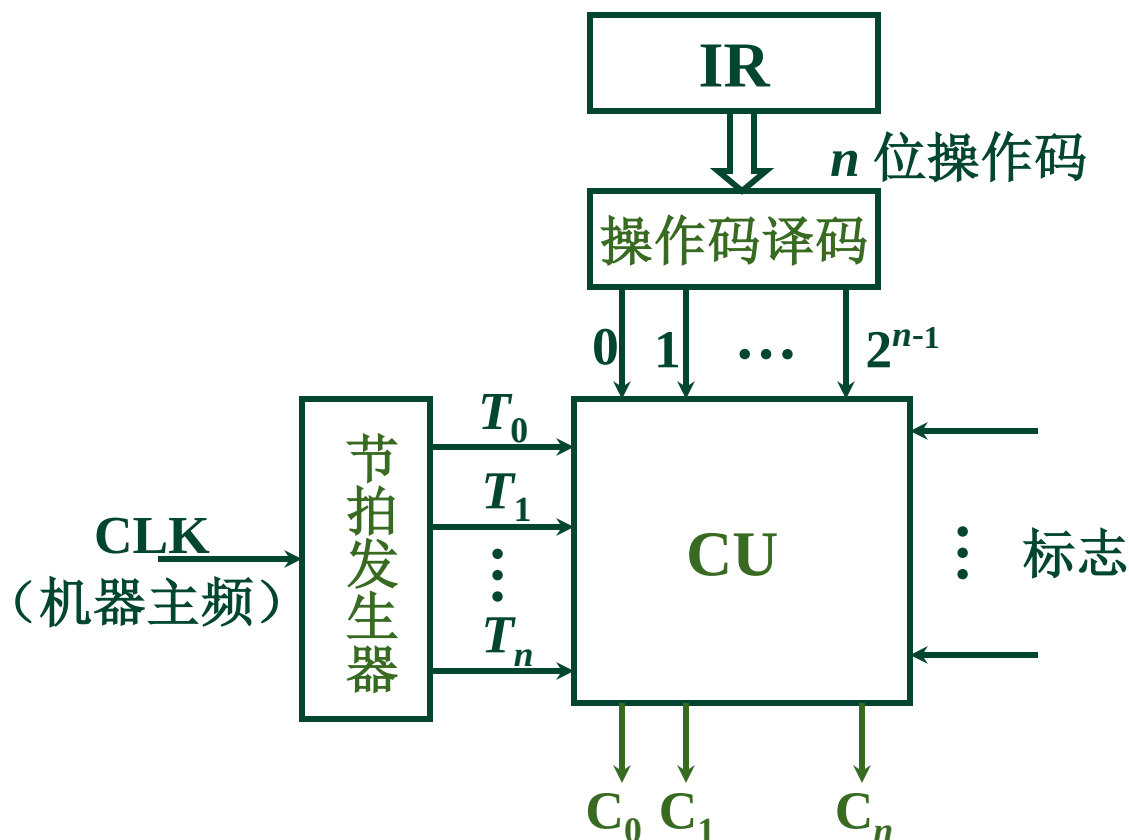
- 控制存储器：微程序存放于存储器中，由于该存储器主要存放控制命令（信号）和下一条执行的微指令地址（简称下址），因此被称为控制存储器。
 - 执行一条指令就是执行一段存放在控制存储器中的微程序。
 - 用**ROM**实现，因为一台计算机指令系统是固定的，所以微程序是固定的，所以控制存储器可以用**ROM**实现
- 微周期：执行一条微指令和取出下一条微指令所需时间
 - 通常一个微周期与一个**CPU**周期时间相等
- 相斥性微命令：不能在一个微周期出现的微命令
 - 如读命令和写命令
- 相容性微命令：能在一个微周期出现的微命令。

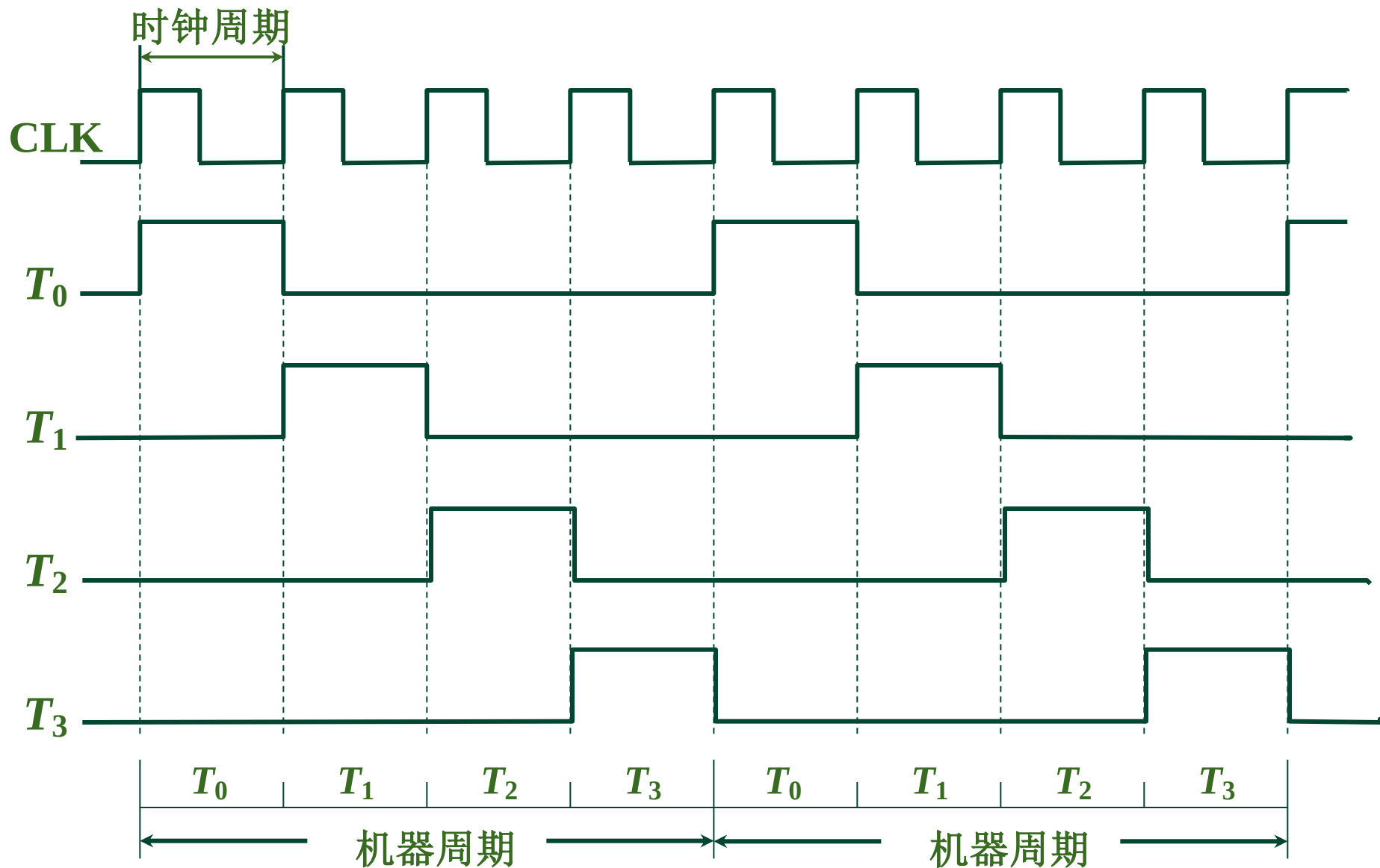
- 微地址寄存器（**CMAR**,或 μ **AR**）用于存放控制存储器的读/写微指令地址。
- 微指令寄存器（ **CMDR**， 或 μ **IR**）用于存放从控制存储器中读出的微指令

10.1 组合逻辑设计

一、组合逻辑控制单元框图

1. CU 外特性



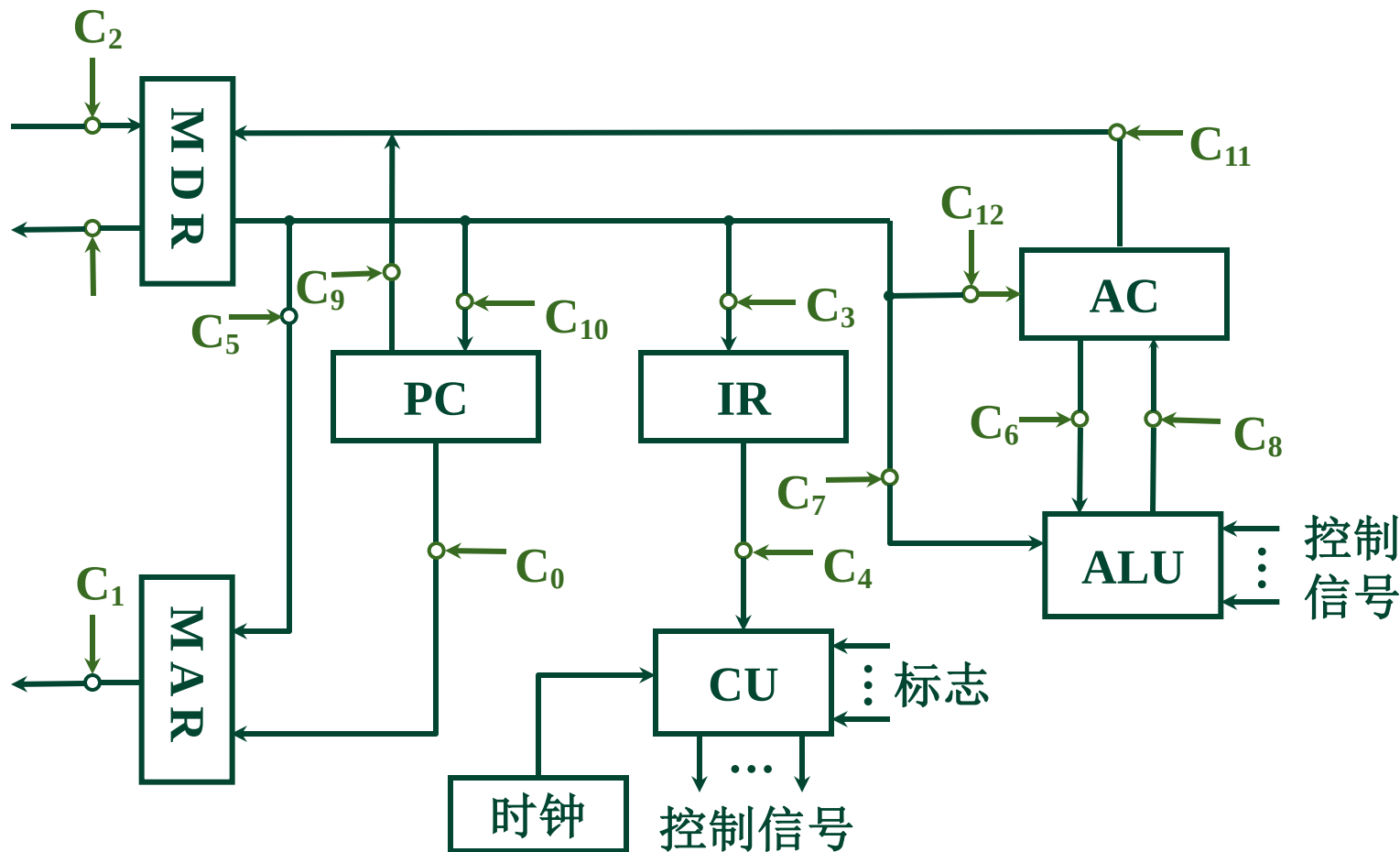


二、微操作的节拍安排

假设：采用 同步控制方式

一个机器周期内有 3 个节拍（时钟周期）

CPU 内部结构采用非总线方式



1. 安排微操作时序的原则

10.1

原则一 微操作的 先后顺序不得 随意 更改

原则二 被控对象不同 的微操作

尽量安排在 一个节拍 内完成

原则三 占用 时间较短 的微操作

尽量 安排在一个节拍 内完成

并允许有先后顺序

2. 取指周期 微操作的 节拍安排

10.1

T_0	$PC \longrightarrow MAR$ $1 \longrightarrow R$	原则二
T_1	$M(MAR) \longrightarrow MDR$ $(PC) + 1 \longrightarrow PC$	原则二
T_2	$MDR \longrightarrow IR$ $OP(IR) \longrightarrow ID$	原则三

3. 间址周期 微操作的 节拍安排

T_0	$Ad(IR) \longrightarrow MAR$ $1 \longrightarrow R$
T_1	$M(MAR) \longrightarrow MDR$
T_2	$MDR \longrightarrow Ad(IR)$

4. 执行周期 微操作的 节拍安排

10.1

① CLA T_0

T_1

$T_2 \quad 0 \longrightarrow AC$

② COM T_0

T_1

$T_2 \quad \overline{AC} \longrightarrow AC$

③ SHR T_0

T_1

$T_2 \quad L(AC) \longrightarrow R(AC)$

$AC_0 \longrightarrow AC_0$

④ CSL

T_0		
T_1		
T_2	$R(AC) \longrightarrow L(AC)$	$AC_0 \longrightarrow AC_n$

⑤ STP

T_0		
T_1		
T_2	$0 \longrightarrow G$	

⑥ ADD X

T_0	$Ad(IR) \longrightarrow MAR$	$1 \longrightarrow R$
T_1	$M(MAR) \longrightarrow MDR$	
T_2	$(AC) + (MDR) \longrightarrow AC$	

⑦ STA X

T_0	$Ad(IR) \longrightarrow MAR$	$1 \longrightarrow W$
T_1	$AC \longrightarrow MDR$	
T_2	$MDR \longrightarrow M(MAR)$	

⑧ LDA X T_0 $Ad (IR) \longrightarrow MAR \quad 1 \longrightarrow R$

T_1 $M (MAR) \longrightarrow MDR$

T_2 $MDR \longrightarrow AC$

⑨ JMP X T_0

T_1

T_2 $Ad (IR) \longrightarrow PC$

⑩ BAN X T_0

T_1

T_2 $A_0 \cdot Ad (IR) + \bar{A}_0 \cdot PC \longrightarrow PC$

5. 中断周期 微操作的 节拍安排

10.1

T_0 $0 \longrightarrow \text{MAR}$ $1 \longrightarrow \text{W}$ 硬件关中断

T_1 $\text{PC} \longrightarrow \text{MDR}$

T_2 $\text{MDR} \longrightarrow \text{M}(\text{MAR})$ 向量地址 $\longrightarrow \text{PC}$

中断隐指令完成

三、组合逻辑设计步骤

- 设计指令的操作码，确定指令长度是固定的还是变长的。
- 确定机器周期、节拍和时钟周期，确定机器周期是固定的还是可变长的。
- 根据指令功能和**CPU**的结构图，绘制每条指令的微操作流程图中并综合成一个总的流程图。
- 给微操作流程图中安排时序，确定每条指令所需的机器周期及在各机器周期需完成的操作，排出**微操作时间表**。
- 根据操作时间表写出微操作的逻辑表达式，即
微操作 = 周期 · 节拍 · 时钟脉冲 · 指令码 · 其他条件
- 根据微操作的表达式，画出组合逻辑电路

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
FE 取指	T_0		$PC \rightarrow MAR$						
			$1 \rightarrow R$						
	T_1		$M(MAR) \rightarrow MDR$						
			$(PC) + 1 \rightarrow PC$						
	T_2		$MDR \rightarrow IR$						
			$OP(IR) \rightarrow ID$						
		I	$1 \rightarrow IND$						
		$\bar{I}$	$1 \rightarrow EX$						

间址特征

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
IND 间址	T_0		Ad (IR) $\rightarrow$ MAR						
			1 $\rightarrow$ R						
	T_1		M(MAR) $\rightarrow$ MDR						
	T_2		MDR $\rightarrow$ Ad (IR)						
		$\overline{\text{IND}}$	1 $\rightarrow$ EX						

间址周期标志

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
EX 执行	T_0		$Ad(IR) \rightarrow MAR$						
			$1 \rightarrow R$						
			$1 \rightarrow W$						
	T_1		$M(MAR) \rightarrow MDR$						
			$AC \rightarrow MDR$						
	T_2		$(AC) + (MDR) \rightarrow AC$						
			$MDR \rightarrow M(MAR)$						
			$MDR \rightarrow AC$						
			$0 \rightarrow AC$						

1. 列出操作时间表

工作 周期 标记	节拍	状态 条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
FE 取指	T_0		$PC \rightarrow MAR$	1	1	1	1	1	1
			$1 \rightarrow R$	1	1	1	1	1	1
	T_1		$M(MAR) \rightarrow MDR$	1	1	1	1	1	1
			$(PC) + 1 \rightarrow PC$	1	1	1	1	1	1
	T_2		$MDR \rightarrow IR$	1	1	1	1	1	1
			$OP(IR) \rightarrow ID$	1	1	1	1	1	1
		I	$1 \rightarrow IND$			1	1	1	1
		$\bar{I}$	$1 \rightarrow EX$	1	1	1	1	1	1

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
IND 间址	T_0		Ad (IR) $\rightarrow$ MAR			1	1	1	1
			1 $\rightarrow$ R			1	1	1	1
	T_1		M(MAR) $\rightarrow$ MDR			1	1	1	1
	T_2		MDR $\rightarrow$ Ad (IR)			1	1	1	1
		$\overline{\text{IND}}$	1 $\rightarrow$ EX			1	1	1	1

三、组合逻辑设计步骤

10.1

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
EX 执行	T_0		$Ad(IR) \rightarrow MAR$			1	1	1	
			$1 \rightarrow R$			1		1	
			$1 \rightarrow W$				1		
	T_1		$M(MAR) \rightarrow MDR$			1		1	
			$AC \rightarrow MDR$				1		
	T_2		$(AC) + (MDR) \rightarrow AC$			1			
			$MDR \rightarrow M(MAR)$				1		
			$MDR \rightarrow AC$					1	
			$0 \rightarrow AC$	1					

2. 写出微操作命令的最简表达式

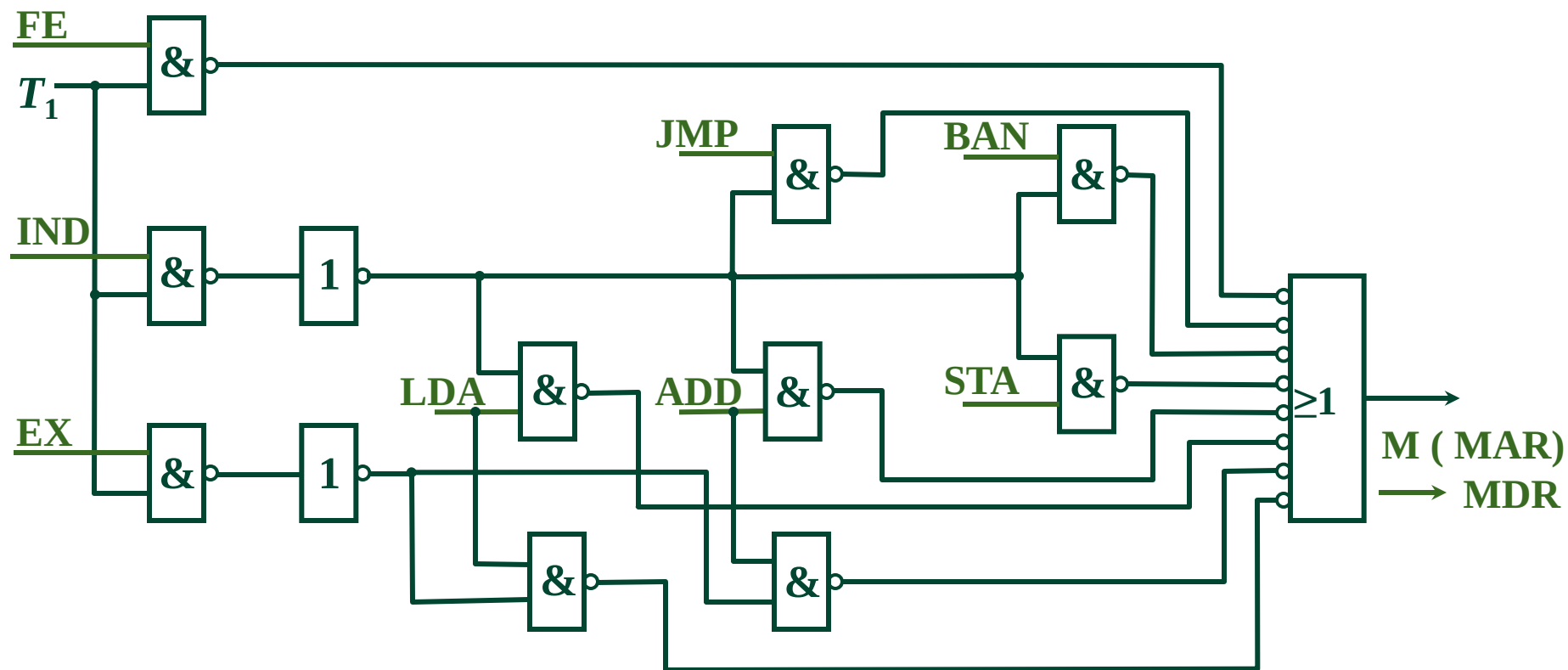
10.1

$$M(MAR) \longrightarrow MDR$$

$$= FE \cdot T_1 + IND \cdot T_1 (ADD + STA + LDA + JMP + BAN) \\ + EX \cdot T_1 (ADD + LDA)$$

$$= T_1 \{ FE + IND (ADD + STA + LDA + JMP + BAN) \\ + EX (ADD + LDA) \}$$

3. 画出逻辑图



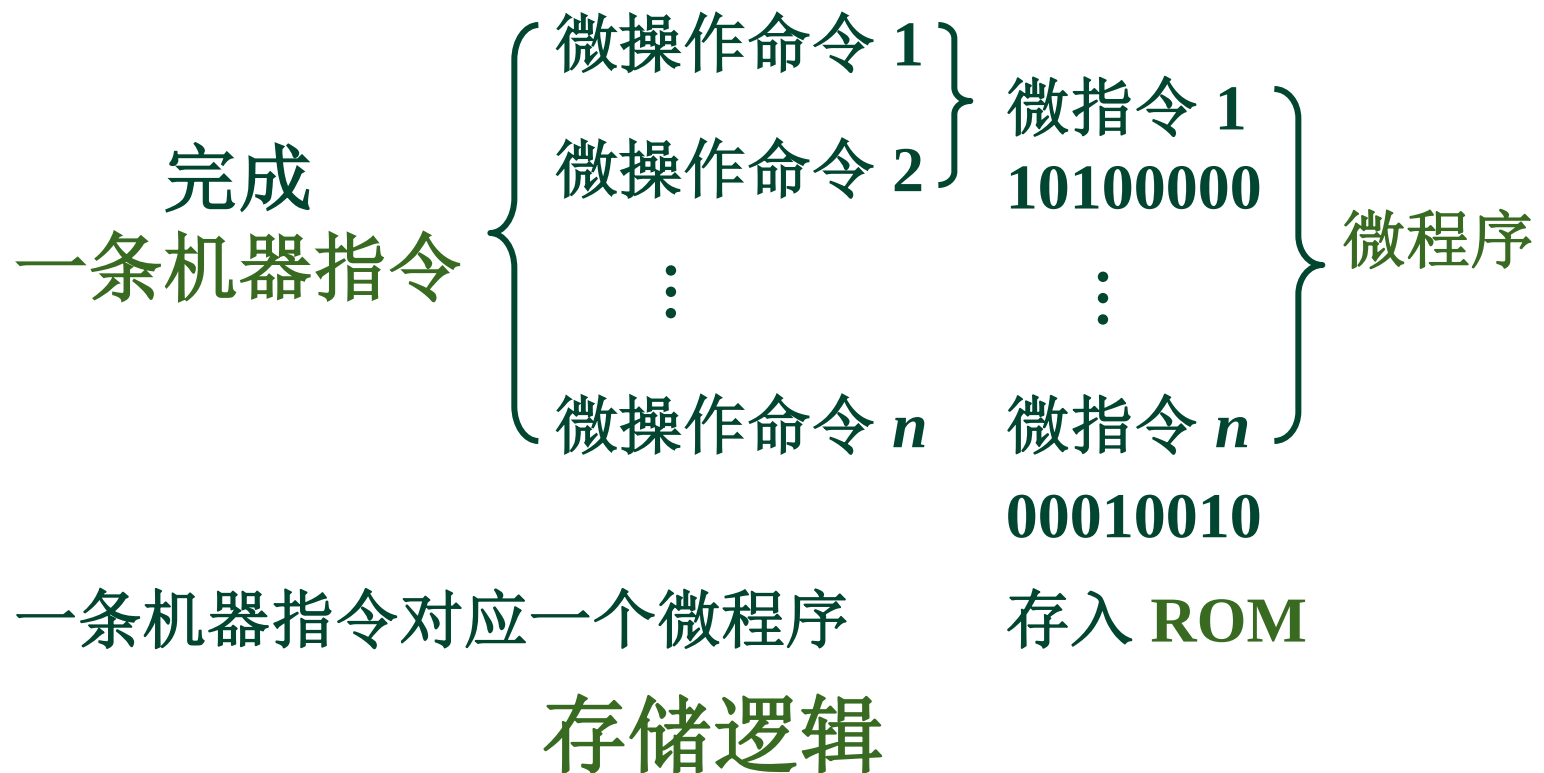
特点

- 思路清晰，简单明了
- 庞杂，调试困难，修改困难
- 速度快 （RISC）

10.2 微程序设计

一、微程序设计思想的产生

1951 英国剑桥大学教授 Wilkes



微指令的格式

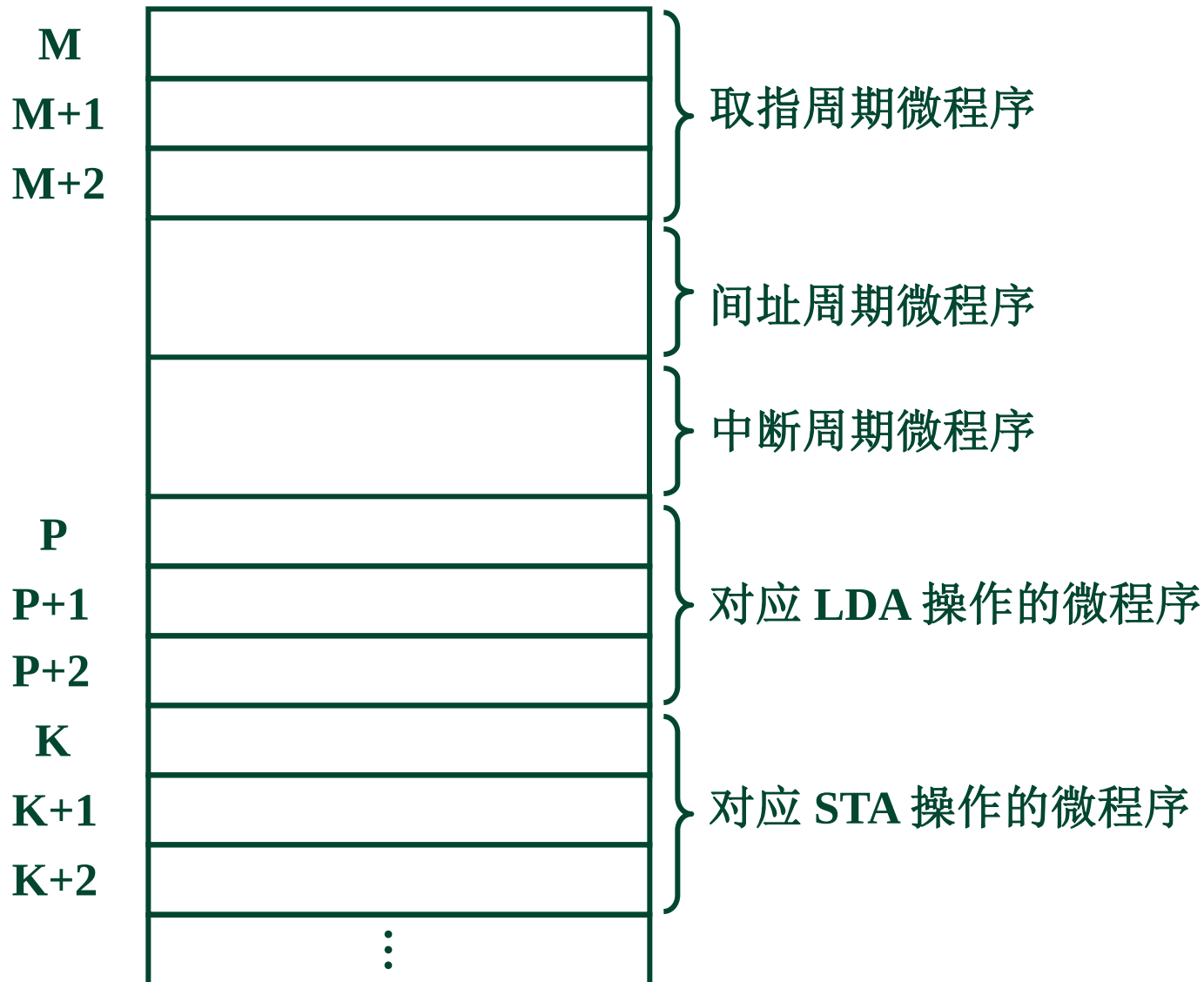
微指令：



- ❖ 控制字段：操作控制，发出各种控制信号
- ❖ 下址字段：顺序控制，指出下条微指令地址，以控制微指令序列的执行顺序

二、微程序控制单元框图及工作原理

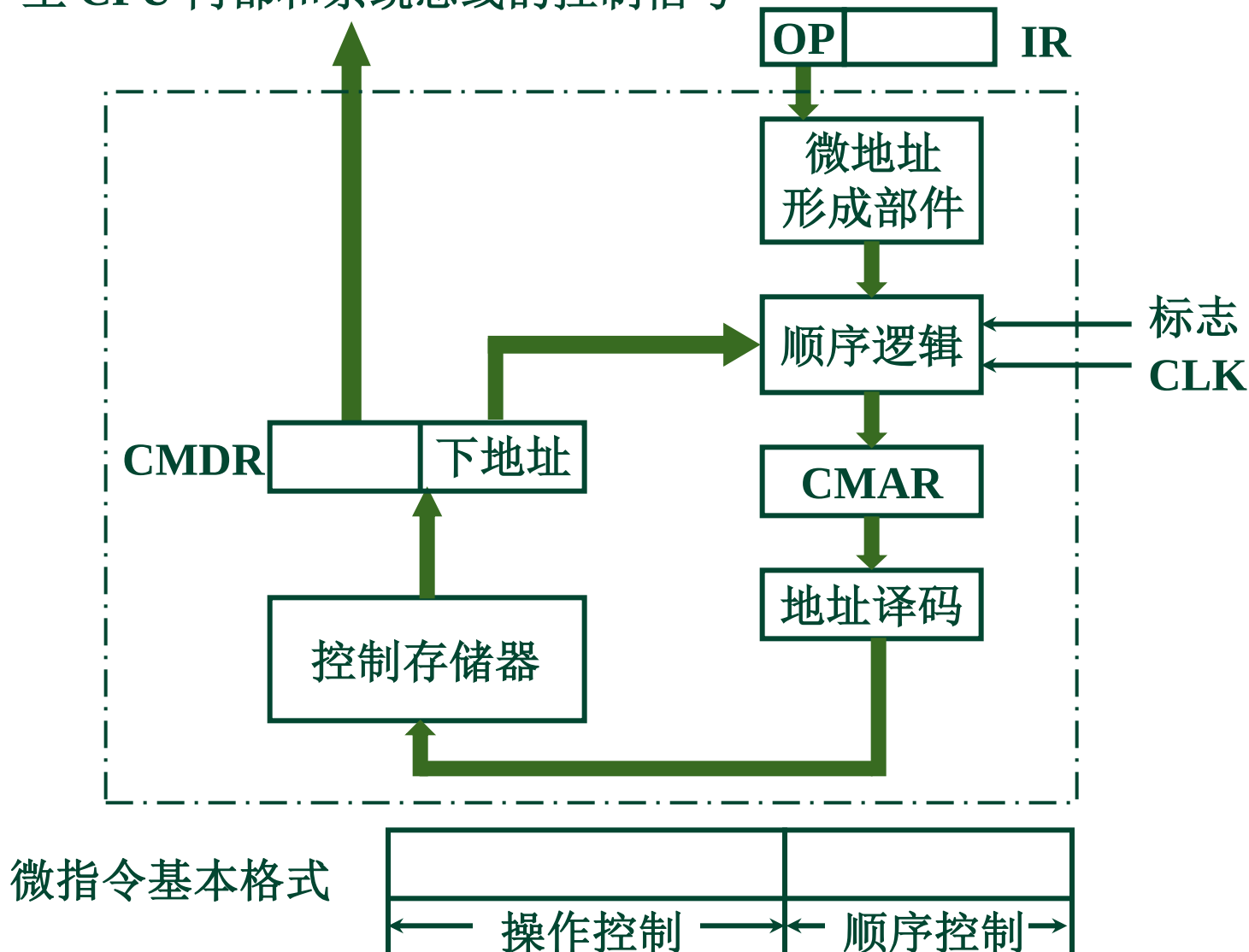
1. 机器指令对应的微程序



2. 微程序控制单元的基本框图

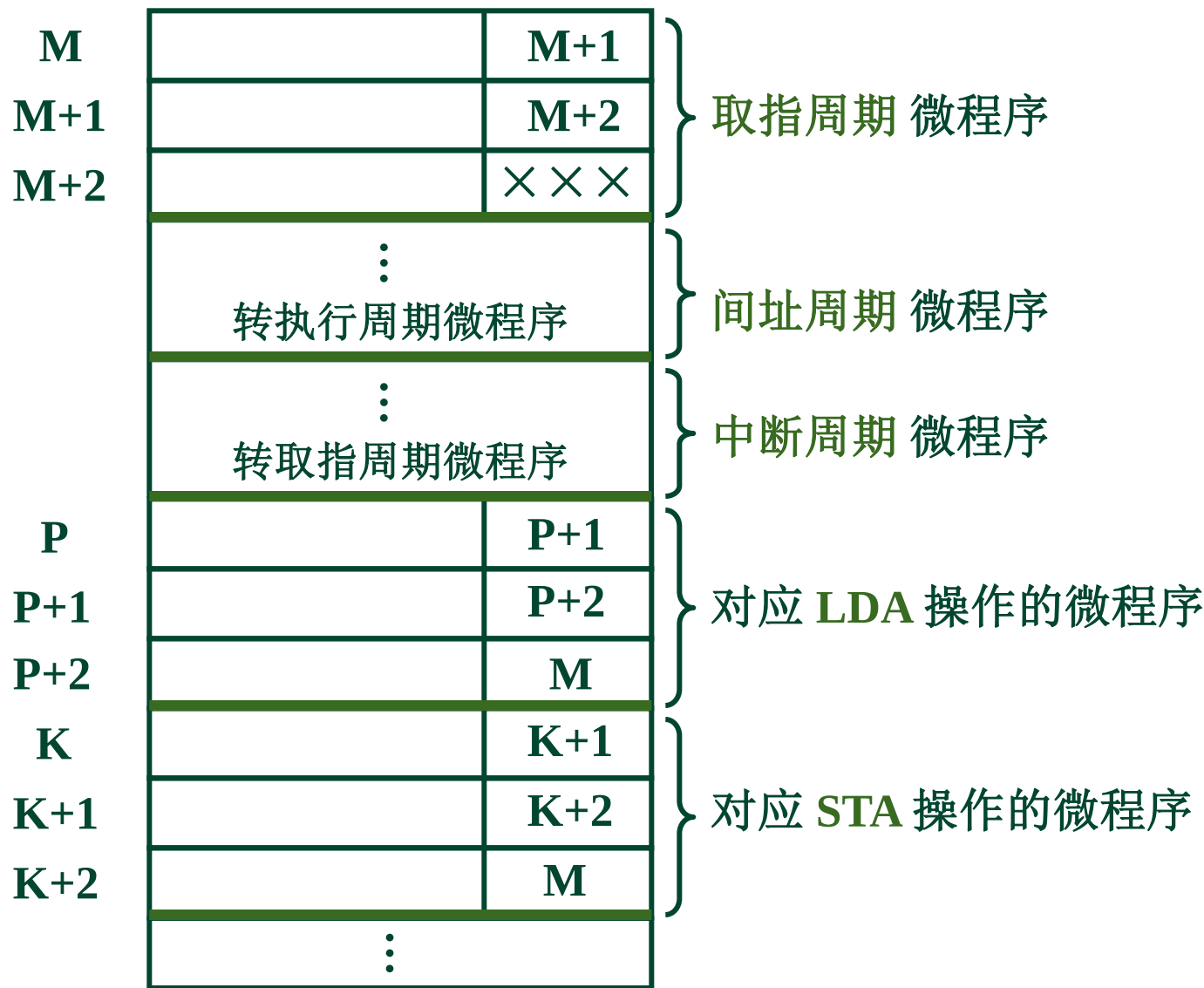
10.2

至 CPU 内部和系统总线的控制信号



二、微程序控制单元框图及工作原理

10.2



3. 工作原理

10.2

控存

主存

用户程序

LDA X
ADD Y
STA Z
STP

M
M+1
M+2

P
P+1
P+2

Q
Q+1
Q+2

K
K+1
K+2

	M+1
	M+2
	× × ×
⋮	
	P+1
	P+2
	M
⋮	
	Q+1
	Q+2
	M
⋮	
	K+1
	K+2
	M
⋮	

取指周期
微程序

对应 LDA 操
作的微程序

对应 ADD 操
作的微程序

对应 STA 操
作的微程序

(1) 取指阶段 执行取指微程序

$M \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令

形成下条微指令地址 $M + 1$

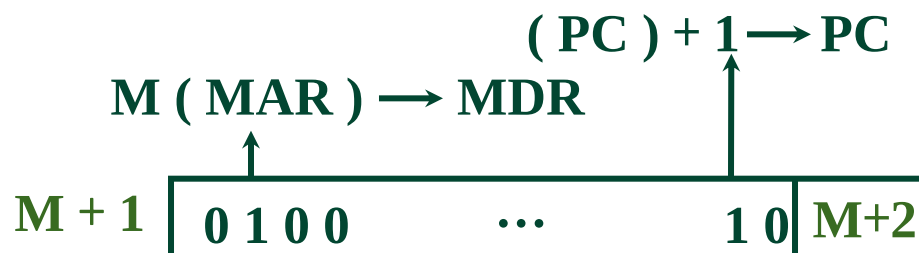


$Ad(CMDR) \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令

形成下条微指令地址 $M + 2$



$Ad(CMDR) \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令



(2) 执行阶段 执行 LDA 微程序

10.2

$OP(IR) \rightarrow \text{微地址形成部件} \rightarrow CMAR \quad (P \rightarrow CMAR)$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令

$Ad(IR) \rightarrow MAR \quad 1 \rightarrow R$



形成下条微指令地址 $Ad(CMDR) \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令

$M(MAR) \rightarrow MDR$



形成下条微指令地址 $Ad(CMDR) \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令

$MDR \rightarrow AC$



形成下条微指令地址 $Ad(CMDR) \rightarrow CMAR$

$(M \rightarrow CMAR)$

$M \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令

⋮

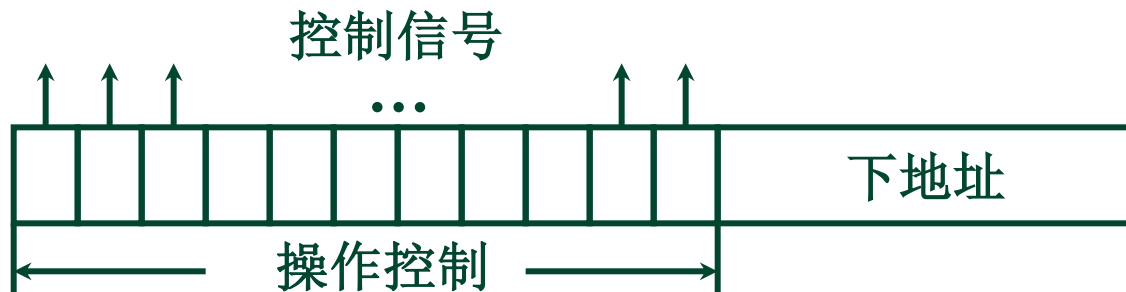


全部微指令存在 CM 中，程序执行过程中 只需读出

- 关键
- 微指令的 操作控制字段如何形成微操作命令
 - 微指令的 后续地址如何形成

1. 直接编码（直接控制）方式

在微指令的操作控制字段中，
每一位代表一个微操作命令

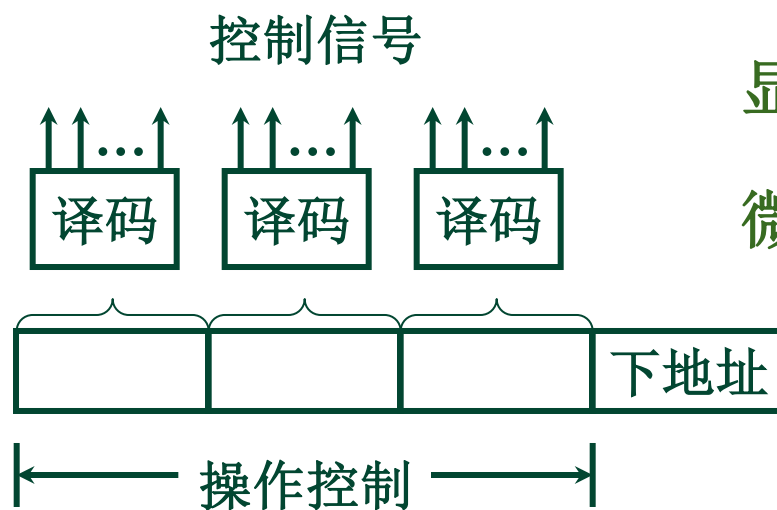


速度最快

某位为 “1” 表示该控制信号有效

2. 字段直接编码方式

将微指令的控制字段分成若干“段”，
每段经译码后发出控制信号



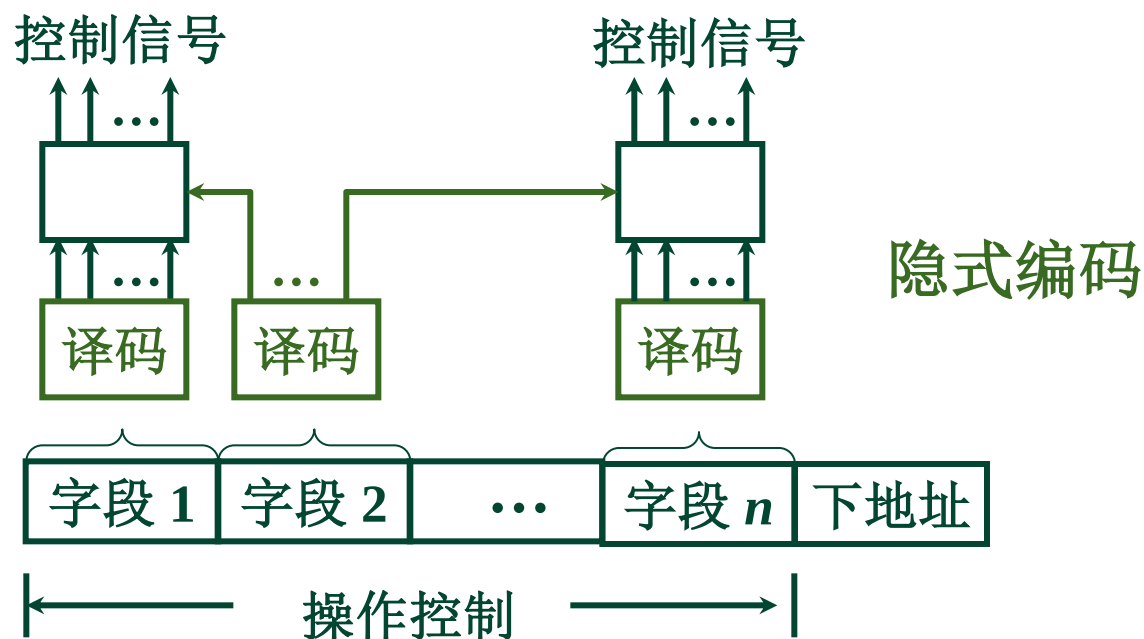
显式编码

微程序执行速度较慢

每个字段中的命令是 互斥 的

缩短 了微指令 字长，增加 了译码 时间

3. 字段间接编码方式



4. 混合编码

直接编码和字段编码（直接和间接）混合使用

5. 其他

1. 微指令的 下地址字段 指出(断定方式)
2. 根据机器指令的 操作码 形成：根据机器指令的操作码，由微地址形成部件形成对应该机器指令微程序的首地址。
3. 增量计数器（顺序地址）

$$(CMAR) + 1 \rightarrow CMAR$$

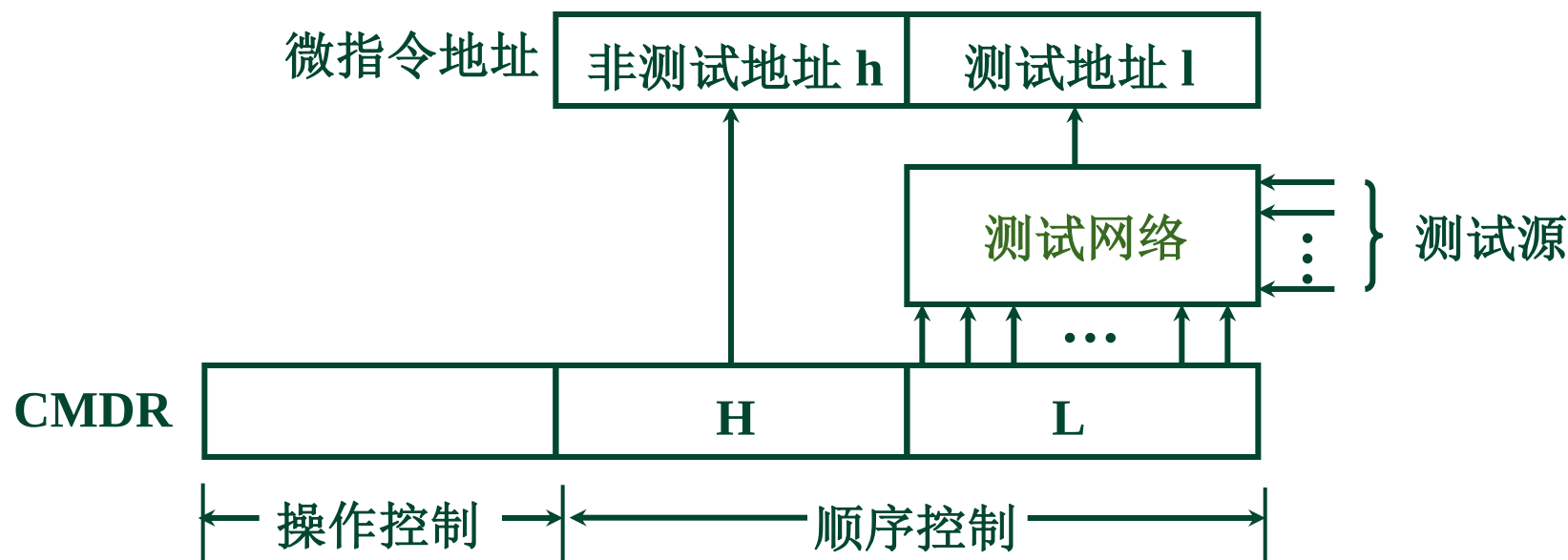
4. 分支转移（转移指令）

操作控制字段	转移方式	转移地址
--------	------	------

转移方式 指明判别条件

转移地址 指明转移成功后的去向

5. 通过测试网络

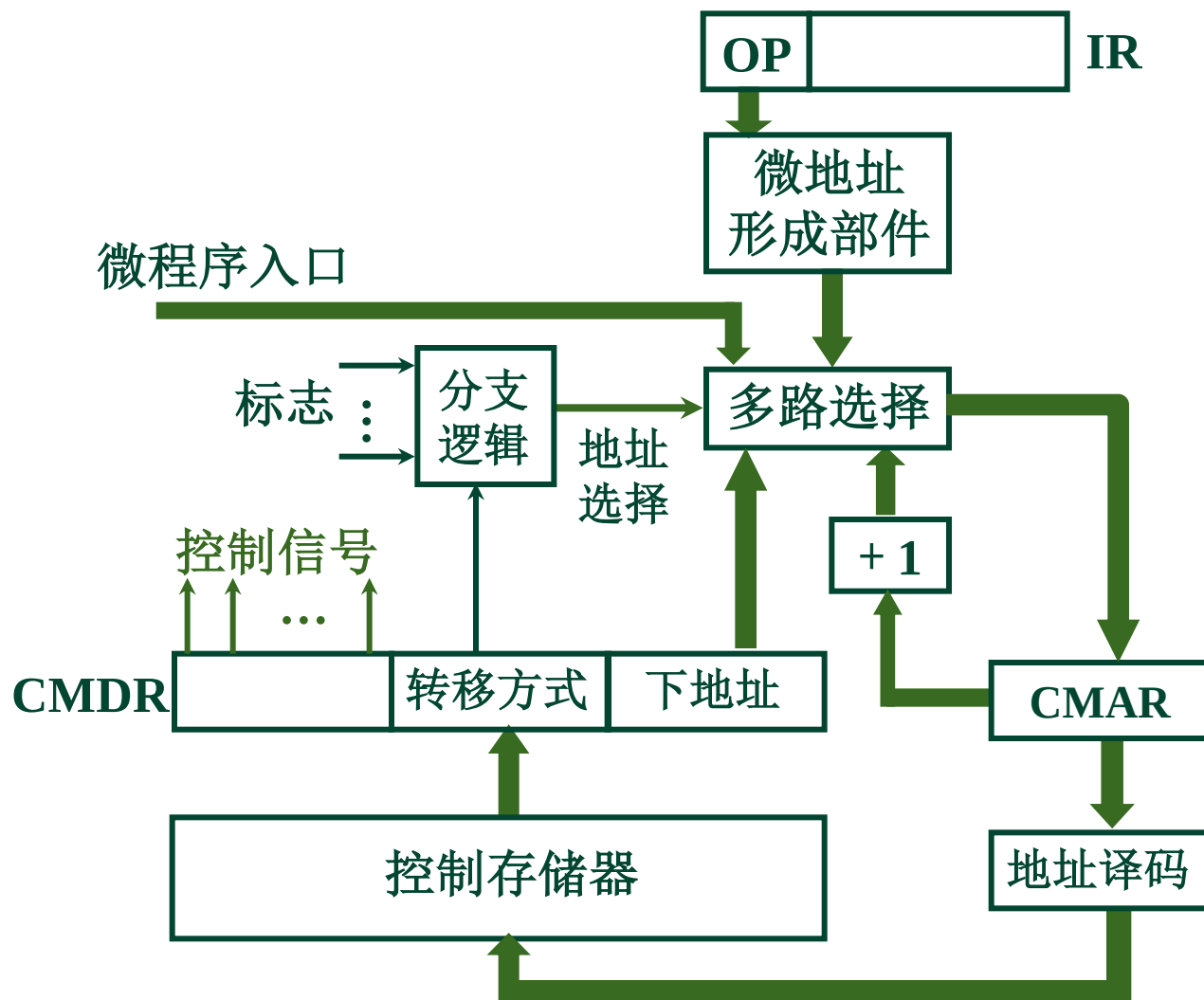


6. 由硬件产生微程序入口地址

加电后，第一条微指令地址 由专门 硬件 产生

中断周期 由 硬件 产生 中断周期微程序首地址

7. 后续微指令地址形成方式原理图



1. 水平型微指令

一次能定义并执行多个并行操作

如 直接编码、字段直接编码、字段间接编码、
直接和字段混合编码

2. 垂直型微指令

类似机器指令操作码 的方式

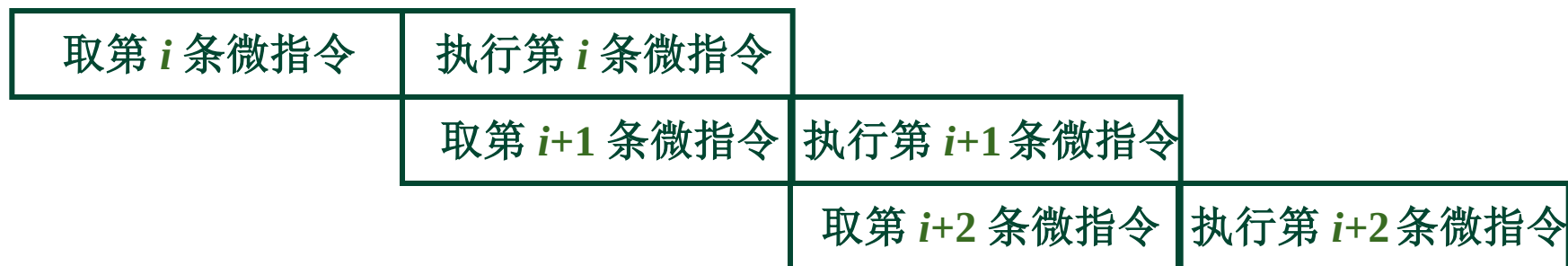
由微操作码字段规定微指令的功能

- (1) 水平型微指令比垂直型微指令 并行操作能力强，
灵活性强
- (2) 水平型微指令执行一条机器指令所要的
微指令 数目少，速度快
- (3) 水平型微指令 用较短的微程序结构换取较长的
微指令结构
- (4) 水平型微指令与机器指令 差别大

串行 微程序控制



并行 微程序控制



1. 写出对应机器指令的微操作及节拍安排

假设 CPU 结构与组合逻辑相同

(1) 取指阶段微操作分析 3 条微指令

T_0 $PC \rightarrow MAR$ $1 \rightarrow R$

T_1 $M(MAR) \rightarrow MDR$ $(PC) + 1 \rightarrow PC$

T_2 $MDR \rightarrow IR$ $OP(IR) \rightarrow$ 微地址形成部件

还需考虑 如何读出 这 3 条微指令 ?

$Ad(CMDR) \rightarrow CMAR$

$OP(IR) \rightarrow$ 微地址形成部件 $\rightarrow CMAR$

(2) 取指阶段的微操作及节拍安排

10.2

考虑到需要 形成后续微指令的地址

T_0 $PC \longrightarrow MAR$ $1 \longrightarrow R$

T_1 $Ad (CMDR) \longrightarrow CMAR$

T_2 $M (MAR) \longrightarrow MDR$ $(PC) + 1 \longrightarrow PC$

T_3 $Ad (CMDR) \longrightarrow CMAR$

T_4 $MDR \longrightarrow IR$ $OP (IR) \longrightarrow$ 微地址形成部件

T_5 $OP (IR) \longrightarrow$ 微地址形成部件 $\longrightarrow CMAR$

考虑到需形成后续微指令的地址

取指微程序的入口地址 M
由微指令下地址字段指出

- 非访存指令

- ① CLA 指令

$T_0 \quad 0 \longrightarrow AC$

$T_1 \quad Ad (CMDR) \longrightarrow CMAR$

- ② COM 指令

$T_0 \quad \overline{AC} \longrightarrow AC$

$T_1 \quad Ad (CMDR) \longrightarrow CMAR$

③ SHR 指令

$$T_0 \quad L(AC) \longrightarrow R(AC) \quad AC_0 \longrightarrow AC_0$$

$$T_1 \quad Ad(CMDR) \longrightarrow CMAR$$

④ CSL 指令

$$T_0 \quad R(AC) \longrightarrow L(AC) \quad AC_0 \longrightarrow AC_n$$

$$T_1 \quad Ad(CMDR) \longrightarrow CMAR$$

⑤ STP 指令

$$T_0 \quad 0 \longrightarrow G$$

$$T_1 \quad Ad(CMDR) \longrightarrow CMAR$$

⑥ ADD 指令

T_0 $Ad (IR) \longrightarrow MAR$ $1 \longrightarrow R$

T_1 $Ad (CMDR) \longrightarrow CMAR$

T_2 $M (MAR) \longrightarrow MDR$

T_3 $Ad (CMDR) \longrightarrow CMAR$

T_4 $(AC) + (MDR) \longrightarrow AC$

T_5 $Ad (CMDR) \longrightarrow CMAR$

⑦ STA 指令

T_0 $Ad (IR) \longrightarrow MAR$ $1 \longrightarrow W$

T_1 $Ad (CMDR) \longrightarrow CMAR$

T_2 $AC \longrightarrow MDR$

T_3 $Ad (CMDR) \longrightarrow CMAR$

T_4 $MDR \longrightarrow M (MAR)$

T_5 $Ad (CMDR) \longrightarrow CMAR$

⑧ LDA 指令

T_0 $Ad (IR) \longrightarrow MAR$ $1 \longrightarrow R$

T_1 $Ad (CMDR) \longrightarrow CMAR$

T_2 $M (MAR) \longrightarrow MDR$

T_3 $Ad (CMDR) \longrightarrow CMAR$

T_4 $MDR \longrightarrow AC$

T_5 $Ad (CMDR) \longrightarrow CMAR$

• 转移类指令

10.2

⑨ JMP 指令

$$T_0 \quad \text{Ad (IR)} \longrightarrow \text{PC}$$

$$T_1 \quad \text{Ad (CMDR)} \longrightarrow \text{CMAR}$$

⑩ BAN 指令

$$T_0 \quad A_0 \cdot \text{Ad (IR)} + \bar{A}_0 \cdot (\text{PC}) \longrightarrow \text{PC}$$

$$T_1 \quad \text{Ad (CMDR)} \longrightarrow \text{CMAR}$$

全部微操作 20个

微指令 38条

(1) 微指令的编码方式

采用直接控制

(2) 后续微指令的地址形成方式

由机器指令的操作码通过微地址形成部件形成

由微指令的下地址字段直接给出

(3) 微指令字长

由 20 个微操作

确定 操作控制字段 最少 20 位

由 38 条微指令

确定微指令的 下地址字段 为 6 位

微指令字长 可取 $20 + 6 = 26$ 位

(4) 微指令字长的确定

10.2

38 条微指令中有 19 条

是关于后续微指令地址 $\longrightarrow$ CMAR

其中 $\left\{ \begin{array}{ll} 1 \text{ 条} & OP(IR) \longrightarrow \text{微地址形成部件} \longrightarrow \text{CMAR} \\ 18 \text{ 条} & Ad(CMDR) \longrightarrow \text{CMAR} \end{array} \right.$

若用 $Ad(CMDR)$ 直接送控存地址线

则省去了输至 CMAR 的时间，省去了 CMAR

同理 $OP(IR) \longrightarrow \text{微地址形成部件} \longrightarrow \text{控存地址线}$

可省去 19 条微指令，2 个微操作

$$38 - 19 = 19$$

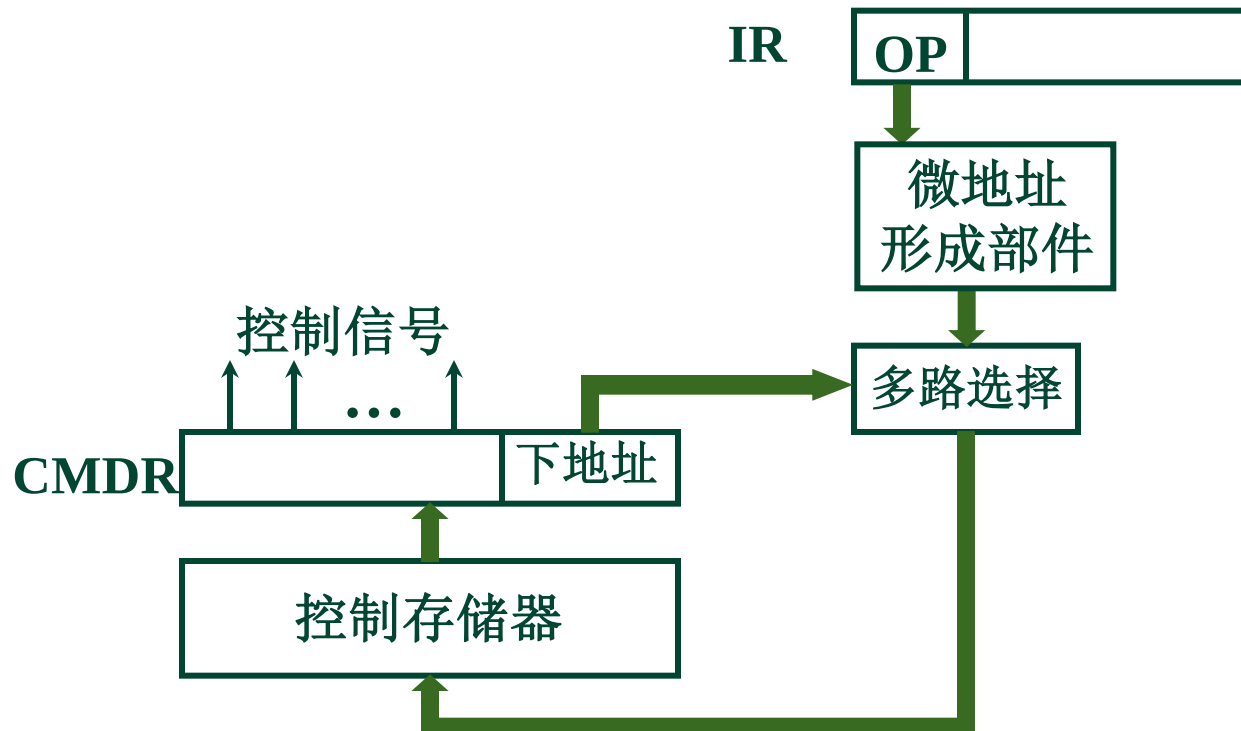
$$20 - 2 = 18$$

下地址字段最少取 5 位

操作控制字段最少取 18 位

(5) 省去了 CMAR 的控制存储器

10.2



考虑留有一定的余量

取操作控制字段
下地址字段

18 位 → 24 位
5 位 → 6 位 } 共 30 位

(6) 定义微指令操作控制字段每一位的微操作



3. 编写微指令码点

10.2

微程序 名称	微指令 地址 (八进制)	微指令（二进制代码）															
		操作控制字段										下地址字段					
		0	1	2	3	4	...	10	...	23	24	25	26	27	28	29	
取指	00	1	1	PC+1→PC						0	0	0	0	0	1		
	01			1	1	MDR→IR						0	0	0	1	0	
	02					1							×	×	×	×	×
CLA	03					Ad(IR)→MAR						0	0	0	0	0	
COM	04			1→R								0	0	0	0	0	
ADD	10		1	M(MAR)→MDR						0	0	1	0	0	1		
	11			1							0	0	1	0	1	0	
	12			1→R								0	0	0	0	0	0
LDA	16		1							1	0	0	1	1	1	1	
	17			1	Ad(IR)→MAR						0	1	0	0	0	0	
	20			M(MAR)→MDR						0	0	0	0	0	0		



Thank You!

Computer Architecture Research Institute · Computer Organization

